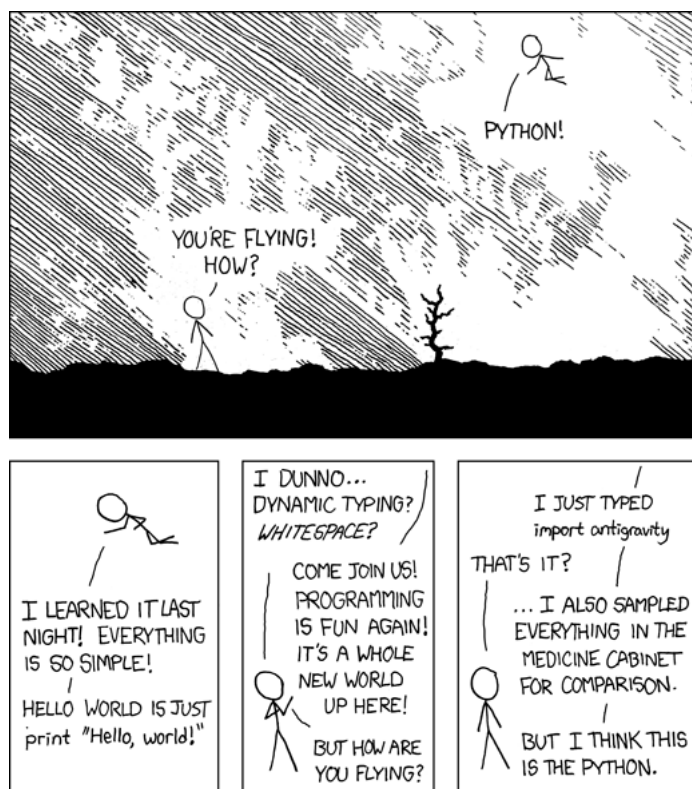


MPI* Maths

Préparation Centrale

Oral Maths-Python



Olivier Caffier - Pierre Bodet



Mis à jour le 14 juin 2025

0 Préambule

Ce polycopié contient à peu près tous les exercices proposés par Monsieur Pascal Vallaëys, professeur de Mathématiques en MPI* au lycée Faidherbe, dans le cadre de la préparation aux oraux Maths-Informatique du concours Centrale-Supélec. Ce travail a été réalisé par des élèves dans un temps limité donc n'est évidemment pas exempté d'erreurs, nous vous invitons à nous le signaler le cas échéant.

Les modules utilisés dans l'entête de nos fichiers sont les suivants :

```
1 # Importation des modules - Olivier
2
3 import numpy as np
4 import numpy.linalg as alg
5 import numpy.random as rd
6 from numpy.polynomial import Polynomial
7
8
9 import math
10 import matplotlib.pyplot as plt
11 from mpl_toolkits.mplot3d import Axes3D
12
13
14 import scipy.optimize as resol
15 import scipy.integrate as integr
```

```
1 # Importation des modules - Pierre
2 https://pastebin.com/7sQp2Qby
```

1 Algèbre linéaire

Exercice 1 On considère la matrice $M = \begin{pmatrix} 0 & x & 0 \\ t & A & y \\ 0 & z & 0 \end{pmatrix}$, où $x = (x_1, \dots, x_n) \in M_{1,n}(\mathbb{R})$, $z = (z_1, \dots, z_n) \in M_{1,n}(\mathbb{R})$, $y = \begin{pmatrix} y_1 \\ \vdots \\ y_n \end{pmatrix} \in$

$M_{n,1}(\mathbb{R})$ et $t = \begin{pmatrix} t_1 \\ \vdots \\ t_n \end{pmatrix} \in M_{n,1}(\mathbb{R})$. La matrice A est telle que chaque coefficient est la moyenne de ses quatre voisins.

1. Montrer que la donnée de A ne définit pas x,y,z et t de manière unique.
2. Montrer que la donnée de x,y,z et t définit au plus une matrice A.
3. Ecrire une fonction python qui a une telle matrice M (mais ne vérifiant pas forcément la condition de moyenne) associe

$$M' = F(M) = \begin{pmatrix} 0 & x & 0 \\ t & A' & y \\ 0 & z & 0 \end{pmatrix} \text{ où } a'_{i,j} = \frac{1}{4} (a_{i-1,j} + a_{i+1,j} + a_{i,j-1} + a_{i,j+1}).$$

4. On définit la suite (M_p) par $M_0 = M$ puis $M_{p+1} = F(M_p)$. Ecrire une fonction qui à A et p associe M_p . Tester pour n=3 ou 4 et p=100 puis 1000. Que peut-on conjecturer.
5. Prouver la conjecture.

Corrigé 1

3. On a le code suivant :

```
1 def associe_prime(m) :
2     taille = m.shape
3     res = m.copy()
4     for i in range(1,taille[0]-1) :
5         for j in range(1,taille[1]-1) :
6             res[i][j] = (1/4) * (m[i-1][j] + m[i+1][j] + m[i][j-1] + m[i][j+1])
7     return res
```

4. De même :

```
1 def f_exo1(m, p):
2     ind_tmp = 0
3     tmp = m.copy()
4     while(ind_tmp < p) :
5         ind_tmp+=1
6         tmp = associe_prime(m)
7     return tmp
```

2 Suites et séries numériques

Exercice 2 Les questions précédées d'un [PY] doivent être répondues à l'aide d'un script Python.

Pour $a > 1$, on définit la suite (u_n) suivante par les conditions :

$$u_0 = a \text{ et } \forall n \in \mathbb{N}, u_{n+1} = u_n + \frac{1}{\sqrt{\ln u_n}}$$

- [PY] Ecrire une fonction qui a pour paramètre a et n et qui renvoie la liste des $n+1$ éléments : $(\frac{u_0}{1}, \frac{u_1}{2} \cdot \ln \sqrt{u_1}, \dots, \frac{u_n}{n+1} \cdot \ln \sqrt{u_n})$.
Pour $a = 2$, donner $\frac{u_{200}}{201} \cdot \ln \sqrt{u_{200}}$
- [PY] Tracer pour $n = 100$ et $a = 2$:
 $(k, \frac{u_k}{k+1} \cdot \ln \sqrt{u_k}), k \in \{0, 1, \dots, 100\}$
Que conjecturer?
- Montrer que (u_n) est bien définie et tend vers $+\infty$.
- Montrer que pour $y > x > 1$ on a :
$$\sqrt{\ln y} - \sqrt{\ln x} \leq \frac{y-x}{2x\sqrt{\ln x}}$$
- En déduire la limite quand n tend vers $+\infty$ de $u_n(\sqrt{\ln u_{n+1}} - \sqrt{\ln u_n})$
- On pose pour $n \in \mathbb{N}, b_n = u_n \sqrt{\ln u_n}$. Montrer que $\lim_{n \rightarrow \infty} (b_{n+1} - b_n) = 1$
- En déduire un équivalent de (b_n) .
- Trouver un équivalent de $\ln u_n$ puis de u_n .
- A quelle condition sur $\alpha \in \mathbb{R}$ la série de terme général $a_n = \frac{1}{u_n \ln n^\alpha}$ converge-t-elle?
- [PY] On appelle $u_n^{(a)}$ la suite définie par $u_0 = a$ et la relation de récurrence de l'énoncé.
Pour $a \in \{2, 4, 6, \dots, 40\}$ et $k \in \{0, \dots, 100\}$, tracer les $u_k^{(a)}$.

Corrigé 2

- On a le code ci-dessous :

```
1 def genere_un(a,n) :
2     # genere les u_n
3     res=[a]
4     ind = 0
5     while ind < n :
6         ind +=1
7         new_u = res[-1] + 1/(np.sqrt(np.log(res[-1])))
8         res.append(new_u)
9     return res
10
11
12 def exo2(a,n) :
13     liste_u = genere_un(a,n)
14     res=[a]
15     for i in range(1,n+1):
16         tmp = (liste_u[i] / (i+1) ) * np.log(np.sqrt(liste_u[i]))
17         res.append(tmp)
18     return res
19
20
21 x = range(101)
22 y = np.array(exo2(2,100))
23
24
25 plt.plot(x, y, marker='o', markersize=3, linestyle='-', color='blue')
26 plt.xlabel("k")
27 plt.ylabel(r"$\frac{u_k}{k+1} \cdot \ln(\sqrt{u_k})$")
28 plt.grid(True)
29 plt.show()
```

- et le graphique suivant :

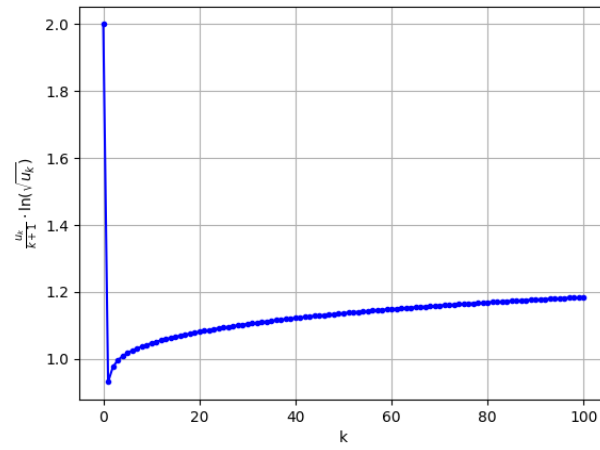


FIGURE 1 – Tracé de la suite demandée

10. ABSL

Exercice 3

1. Soit $(u_n)_{n \in \mathbb{N}^*}$ une suite strictement croissante à termes dans $\mathbb{N}^*$ vérifiant $\lim_{n \rightarrow +\infty} u_{n+1} - u_n = +\infty$.

Montrer qu'il existe une unique bijection $\sigma : \mathbb{N}^* \rightarrow \mathbb{N}^*$ vérifiant :

- (a) $\forall n \in \mathbb{N}^*, \sigma(2n) = u_n$
- (b) La suite $(\sigma(2n+1))_{n \in \mathbb{N}}$ est strictement croissante

2. PYTHON

Pour $\alpha \in \mathbb{R}$, on pose pour tout $n \in \mathbb{N}^* : u_n = \lfloor n^\alpha \rfloor$.

On a en particulier $u_1 = 1$.

On admet que $(u_n)_{n \in \mathbb{N}^*}$ vérifie toutes les conditions de la question 1, et on note σ_α la bijection définie à cette question.

Ecrire une fonction prenant (n, α) en argument et renvoyant la liste $L = [\sigma_\alpha(1), \sigma_\alpha(3), \dots, \sigma_\alpha(2n+1)]$.

Indication : La fonction `math.floor` du module `math` calcule la partie entière du nombre donné en argument.

3. Pour toute bijection $\sigma : \mathbb{N}^* \rightarrow \mathbb{N}^*$, on note $\varepsilon = \{ \beta \in \mathbb{R} \mid \sum_{n \geq 1} \frac{\sigma(n)}{n^\beta} \text{ converge} \}$. Donner un exemple de bijection pour laquelle ε est vide.
4. On suppose que pour une bijection $\sigma : \mathbb{N}^* \rightarrow \mathbb{N}^*$, ε est non vide. Montrer que ε est un intervalle non borné ne contenant pas 2.
5. On considère la suite $(u_n)_{n \in \mathbb{N}^*}$ définie à la question 2. Montrer qu'elle vérifie les hypothèses de la question 1.

```
1 def u(n,a):
2     return np.floor(n**a)
3
4 def sigma(n,a):
5     sigma_impairs = []
6     indice_u = 1
7     indice = 1
8     while len(sigma_impairs)<n: # (un) est croissante donc il suffit de parcourir ses elements dans l'ordre
9         if u(indice_u,a) == indice:
10             indice_u += 1
11             indice += 1 # sigma(2*indice_u) = indice donc il ne peut pas etre present
12         elif u(indice_u,a) < indice:
13             indice_u += 1
14         else: #u(indice_u,a) > indice
15             sigma_impairs.append(indice) # indice n'est pas un antecedent de u
16             indice += 1
17     return np.array(sigma_impairs)
```

Exercice 4 On considère une suite (u_n) de points de $[0,1]$.

Pour $0 \leq a < b \leq 1$, on pose $S_n(a, b) = \text{card}\{k \in [1, n] / u_k \in [a, b]\}$

On dit que (u_n) est équirépartie si $\frac{S_n(a,b)}{n} \rightarrow b-a \forall 0 \leq a < b \leq 1$

1. Montrer que si (u_n) est équirépartie, alors elle est dense dans $[0,1]$. La réciproque est-elle vraie?
2. Soit $n \in \mathbb{N}^*$. On pose $D_n = \sup_{0 \leq a < b \leq 1} |\frac{S_n(a,b)}{n} - (b-a)|$ et $D_n^* = \sup_{0 < \alpha < 1} |\frac{S_n(0,\alpha)}{n} - \alpha|$
 - (a) Programmer une fonction Python qui renvoie la valeur de $S_n(a, b)$. Tester cette fonction avec les suites $u_n = \sqrt{2}n \text{ modulo } 1$, $v_n = \cos(n)$, $w_n = \frac{1}{2}n \text{ modulo } 1$. On pourra définir une fonction `modulo1(r)`.
 - (b) Programmer une fonction Python qui renvoie D_n . Pour estimer la borne supérieure, on pourra générer des couples (a,b) dans $[0,1]$ et garder le maximum de ces valeurs.
 - (c) Montrer que $D_n^* \leq D_n \leq 2D_n^*$.
3. Montrer que (u_n) est équirépartie si et seulement si D_n tend vers 0.

Corrigé 4

1. • Montrons le résultat par contraposée :

$$(u_n) \text{ pas dense dans } [0,1] \implies (u_n) \text{ pas équirépartie}$$

Soit donc (u_n) une suite qui n'est pas dense dans $[0,1]$: il existe donc $a < b \in [0,1]$ tels que $\forall n \in \mathbb{N}, u_n \notin [a, b[$.
Dès lors,

$$\lim_{n \rightarrow +\infty} \frac{S_n(a, b)}{n} = 0$$

car $S_n(a, b) = 0$ pour tout $n \in \mathbb{N}$.

Dès lors cette suite n'est pas équirépartie, d'où le résultat par contraposée.

- La réciproque est évidemment fausse : soit $(u_n) \in [0,1]^{\mathbb{N}}$ une suite équirépartie (et donc dense d'après notre démonstration). On construit la suite (v_n) comme suit :

$$\begin{cases} v_{2n} = u_n \\ v_{2n+1} = 0 \end{cases}$$

qui n'est plus équirépartie du tout (on vient de coller la moitié des termes en 0)! Par contre on garde la densité, par densité de (u_n) :

2. (a) On s'intéresse même ici à l'équirépartition modulo 1, on pourra penser au critère de Weyl plus tard! La fonction modulo 1 vient assez instinctivement :

```

1 def modulo_1(r) :
2     return r - np.floor(r)
3
4
5 def in_bornes(x,a,b) :
6     return (x>=a) & (x<b)
7
8
9 def calcule_s_n(suite_u,a,b) :
10    tmp = [u for u in suite_u if (u >= a) & (u < b)]
11    return len(tmp)
12
13
14 n = 2000
15
16 suite_1 = [modulo_1(np.sqrt(2) * i) for i in range(1,n+1)]
17 suite_2 = [np.cos(i) for i in range(1,n+1)]
18 suite_3 = [modulo_1(1/2 * i) for i in range(1,n+1)]

```

et en faisant des tests, on retrouve le critère de Weyl (son corollaire) :

Corollaire du critère de Weyl

Soit $\alpha \in \mathbb{R}$, on a équivalence entre :

- $(\alpha n \bmod 1)_n$ est équirépartie;
- α est irrationnel;

(b) Voilà :

```
1 def trouve_dn(suite_u,n) :  
2     tab_a = np.arange(0,1,0.01)  
3     max = 0.  
4     for a in tab_a :  
5         sub_tab_b = np.arange(a,1,0.01)  
6         for b in sub_tab_b :  
7             tmp = calcule_s_n(suite_u,a,b)/n - (b-a)  
8             if tmp > max :  
9                 max = tmp  
10    return max
```

Exercice 5 Soit une suite $(u_n)_{n \in \mathbb{N}}$.

On pose, pour tout $n \in \mathbb{N}$, $s_n = \sum_{k=0}^n u_k$ et, pour tout $I \subset \mathbb{N}$ de cardinal fini, $S_I = \sum_{k \in I} u_k$.

La somme vide est considérée nulle ($S_\emptyset = 0$).

Pour tout $n \in \mathbb{N}$, on pose $E_n = \{S_I \mid I \in \mathcal{P}(\{0; n\})\}$ et $E = \bigcup_{n \in \mathbb{N}} E_n$.

1. Écrire une fonction Python qui prend en entrée une suite u (définie sous forme de fonction) et un entier naturel n et qui renvoie la liste des valeurs de s_k pour k allant de 0 à n .

2. Écrire une fonction Python qui prend en entrée une suite u (définie sous forme de fonction) et un entier naturel n et qui renvoie la liste des E_k pour k allant de 0 à n .

On remarque que $E_n = E_{n-1} \cup \{x + u_n \mid x \in E_{n-1}\}$.

On pourra dans un premier temps utiliser des listes puis utiliser la fonction `set` qui renvoie des ensembles.

3. En déduire $s_0, \dots, s_{10}$ et $E_0, \dots, E_{10}$, en prenant $u_n = \left\lfloor \left(\frac{3}{2}\right)^n \right\rfloor$. Faire une conjecture sur E_n et sur E .

Corrigé 5 1.

```
1 def s(u,n):
2     valeurs = [u(0)]
3     for k in range(1,n+1):
4         valeurs.append(valeurs[-1] + u(k))
5     return valeurs
```

2.

```
1 def e(u,n):
2     liste_ek = [[u(0)]] # On cree d'abord des listes au lieu d'ensembles pour pouvoir les manipuler
3     facilement
4     for k in range(1,n+1):
5         temp = []
6         for elmt in liste_ek[-1]:
7             temp.append(elmt + u(k))
8         liste_ek.append(liste_ek[-1] + temp) # Concatenation de listes
9     ensemble_ek = []
10    for ek in liste_ek:
11        ensemble_ek.append(set(ek))
12    return ensemble_ek
```

3.

```
1 print(s(lambda n: np.floor((3/2)**n),10))
2 print(e(lambda n: np.floor((3/2)**n),10))
```

Exercice 6 Énoncé identique au précédent

Corrigé 6 Le corrigé est donc le même

3 Fonctions d'une variable réelle

Exercice 7 Pour tout $k \in \mathbb{N}$, on définit pour tout $n > k - 1$: $\sigma_k(n) = \frac{1}{2n+1} \binom{2n+1}{2k+1}$.

1) b) Écrire une fonction python prenant en paramètre un entier p , renvoyant un tableau contenant les polynômes $S_0, \dots, S_p$

On définit Q_p par $Q_0 = 1, Q_1 = S_1$ et pour tout $p \in \mathbb{N}$, $Q_p = (-1)^{p+1} p S_p + \sum_{k=1}^{p-1} (-1)^{k+1} S_k Q_{p-k}$.

(Modérateur : calcul non fait mais je doute fort du p devant S_p)

2) a) Écrire une fonction prenant en paramètre un entier p et renvoyant le polynôme Q_p

b) Écrire une fonction renvoyant une valeur approchée de la somme $\sum_{n=1}^{+\infty} (\frac{2}{\pi n})^{2p}$. La comparer à la valeur du coefficient dominant de Q_p pour p compris entre 1 et 10. Faire une conjecture.

Corrigé 7 1) b)

```
1 def s(k):
2     poly = Polynomial([1])
3     for i in range(2*k+1):
4         poly = poly * Polynomial([1-i,2])
5     return (poly // Polynomial([1,2])) / np.math.factorial(2*k+1) # ATTENTION: pour diviser 2 polynomes: //,
    pour diviser par un scalaire: /
```

2) a)

```
1 def q(p):
2     if p==0:
3         return Polynomial([1])
4     if p==1:
5         return s(1)
6     poly = (-1)**(p+1) * p * s(p)
7     for k in range(1,p):
8         poly = poly + (-1)**(k+1) * s(k) * q(p-k) # Tres mauvaise complexite: on pourrait utiliser de la
    memoisation (difficile de generer q(15) ici)
9     return poly
```

b)

```
1 def somme(k,p):
2     return np.sum(np.array([(2/(np.pi*n))**(2*p) for n in range(1,k+1)]))
3
4 for p in range(1,11):
5     print(somme(1000,p)-q(p).coef[-1])
```

Exercice 8 Écrire une fonction qui à $P \in \mathbb{C}[X]$ renvoie $t \mapsto P(e^{it})$ et tester avec $P = X^2 - X + \frac{1}{2}$.

Corrigé 8 On a la fonction suivante :

```

1 def f_polynome (P) :
2     """
3     Prend un polynome P et renvoie f_P une fonction qui prend t comme variable
4     """
5     return lambda t: P(np.exp(1j * t))
6
7
8 # Définir le polynome P(X) = X^2 - X + 1/2
9 P = Polynomial([1, -1, 0.5])
10
11 # Obtenir la fonction t -> P(e^{it})
12 f = f_polynome(P)
13
14 # Tester pour quelques valeurs de t
15 t_vals = np.linspace(0, 2 * np.pi, 300)
16 f_vals = f(t_vals)
17
18 # Affichage des parties réelle et imaginaire
19 plt.plot(t_vals, f_vals.real, label="Re(P(e^{it}))")
20 plt.plot(t_vals, f_vals.imag, label="Im(P(e^{it}))")
21 plt.title("P(e^{it}) avec P(X) = X^2 - X + 1/2")
22 plt.xlabel("t")
23 plt.ylabel("Valeur complexe")
24 plt.legend()
25 plt.grid(True)
26 plt.show()

```

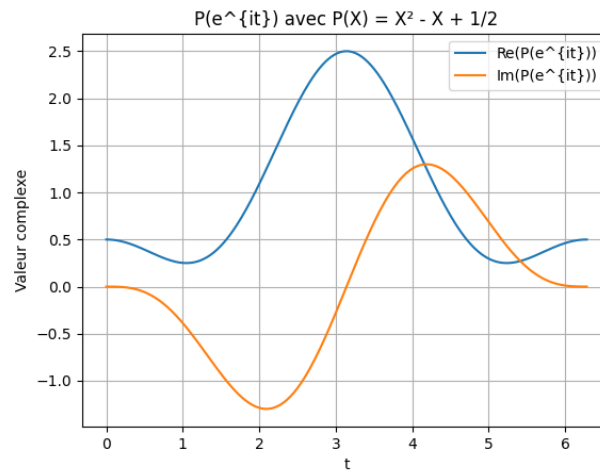


FIGURE 2 – Tracé de $t \mapsto \Re(P(e^{it}))$ et $t \mapsto \Im(P(e^{it}))$

Exercice 9 Soient $H_0 = 1$ et $H_1 = X$ deux polynômes. On définit la famille $(H_k)_{k \in \mathbb{N}}$ par :

$$\forall k \in \mathbb{N}, H_{k+2} = XH_{k+1} - \frac{k+1}{2}H_k$$

On représente un polynôme $a_0 + \dots + a_n X^n$ avec $a_n \neq 0$ par la liste $L = [a_0, \dots, a_n]$ sur Python. On s'interdit le module **Numpy**. **Polynomial**.

1. Coder :

- **Enleve_Zeros(P)** qui enlève de P les 0 inutiles;
- **Mult(a,P)** qui renvoie la liste qui représente aP ;
- **MultX(P)** qui renvoie la liste qui représente XP ;
- **Somme(P,Q)** qui renvoie la liste qui représente $P + Q$;
- **Derive(P)** qui renvoie la liste qui représente P' .

2. Déterminer les dix premiers polynômes de la famille définie. Donner le coefficient dominant et le degré de ces polynômes.

Tracer les H_k pour k allant de 1 à 5 et déterminer une condition que les racines des H_k vérifient.

Corrigé 9

1. On a les fonctions suivantes :

```

1  def Enleve_Zeros(P) :
2      return [b for b in P if b != 0]
3
4  def Mult(a,P) :
5      return [a * b for b in P]
6
7  def MultX(P) :
8      res = P.copy()
9      return [0] + res
10
11 def Somme(P, Q):
12     '''
13     Somme les polynomes P et Q
14     '''
15     n_p, n_q = len(P), len(Q)
16     n = max(n_p, n_q)
17     res = []
18     for i in range(n):
19         coeff_P = P[i] if i < n_p else 0 # Si on depasse la taille de P, il n'y a plus de coeff
20         coeff_Q = Q[i] if i < n_q else 0 #
21         res.append(coeff_P + coeff_Q)
22
23     return res
24
25 def Derive(P) :
26     return [i * P[i] for i in range(1,len(P))] # On evite Enleve_Zeros car sinon on obtiendrait pas P'

```

2. On utilisera une fonction **Genere(n)** qui renverra la liste des n polynômes recherchés (ici $n = 10$).

```

1  def Genere(n) :
2      #Genere les H_k
3      res = [[1], [0,1]]
4      for i in range(2,n) :
5          new_p = Somme(MultX(res[i-1]), Mult(- (i-1)/2, res[i-2]))
6          res.append(new_p)
7      return res
8
9  def Coef_dom(P) :
10     # Donne le coeff dominant d'un polynome
11     return P[len(P)-1]
12
13 def deg(P) :
14     # Donne le degre d'un polynome
15     return len(P) -1
16

```

```

17 # Exemples de H_1 jusqu'e H_10
18 prem_10 = Genere(10)
19 cd_H10, deg_H10 = [Coef_dom(P) for P in prem_10], [deg(P) for P in prem_10]
20
21 def evaluate(P):
22     """
23     Retourne une fonction qui evalue le polynome represente par la liste P
24     (coefficients de degre croissant) sur un scalaire ou un tableau t.
25     """
26     def f(t):
27         t = np.asarray(t)
28         res = np.zeros_like(t, dtype=np.float64)
29         for i in range(len(P)-1, -1, -1): # Horner: du plus haut degre au plus bas
30             res = res * t + P[i]
31         return res
32     return f
33
34 # Trace des fonctions
35 h0 = evaluate(prem_10[0])
36 h1 = evaluate(prem_10[1])
37 h2 = evaluate(prem_10[2])
38 h3 = evaluate(prem_10[3])
39 h4 = evaluate(prem_10[4])
40
41 t_vals = np.linspace(-1,1,300)
42 h0_vals = h0(t_vals)
43 h1_vals = h1(t_vals)
44 h2_vals = h2(t_vals)
45 h3_vals = h3(t_vals)
46 h4_vals = h4(t_vals)
47
48 plt.clf()
49 plt.plot(t_vals,h0_vals, label = r"$H_0(t)$")
50 plt.plot(t_vals,h1_vals, label = r"$H_1(t)$")
51 plt.plot(t_vals,h2_vals, label = r"$H_2(t)$")
52 plt.plot(t_vals,h3_vals, label = r"$H_3(t)$")
53 plt.plot(t_vals,h4_vals, label = r"$H_4(t)$")
54
55 plt.title(r"Trace des premiers $H_k(t)$")
56 plt.xlabel("t")
57 plt.legend()
58 plt.grid(True)
59 plt.show()

```

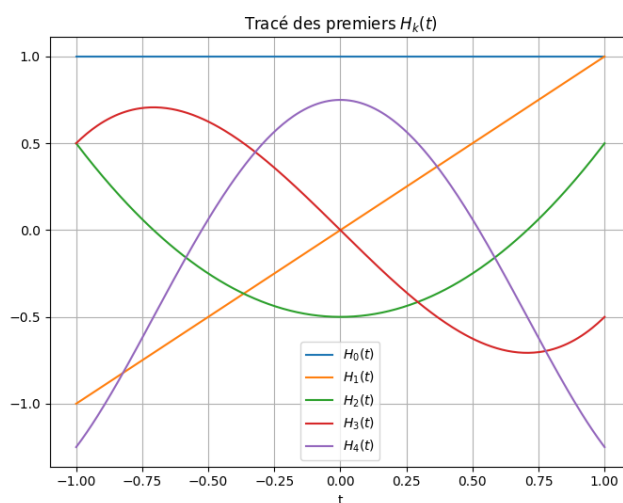


FIGURE 3 – Tracé des 5 premiers H_k

Exercice 10 On considère la suite de polynômes suivante :

$$\begin{cases} T_0 = 1 \\ T_1 = X \\ \forall n \in \mathbb{N}, T_{n+2} = 2XT_{n+1} - T_n \end{cases}$$

1. Écrire en Python une fonction `fonction(n)` telle que pour $n \in \mathbb{N}$, renvoie le polynôme T_n . On utilisera la classe `Polynomial` du module `numpy.polynomial`.
2. Tracer T_1, T_2, T_3, T_4, T_5 . Combien de racines ont-ils ?
3. Vérifier (toujours avec Python) sur quelques valeurs de $n, m \in \mathbb{N}$ que $T_n \circ T_m = T_m \circ T_n$

Corrigé 10

1. On a la fonction suivante :

```

1 def fonction(n) :
2     if n == 0 :
3         return Polynomial([1])
4     elif n == 1 :
5         return Polynomial([0,1])
6     else :
7         t_minus_2 = Polynomial([1])
8         t_minus_1 = Polynomial([0,1])
9         ind = 2
10        while ind <= n :
11            t_tmp = 2 * Polynomial([0,1]) * t_minus_1 - t_minus_2
12            ind += 1
13            t_minus_2 = t_minus_1
14            t_minus_1 = t_tmp
15        return t_minus_1

```

2. On a l’affichage suivant :

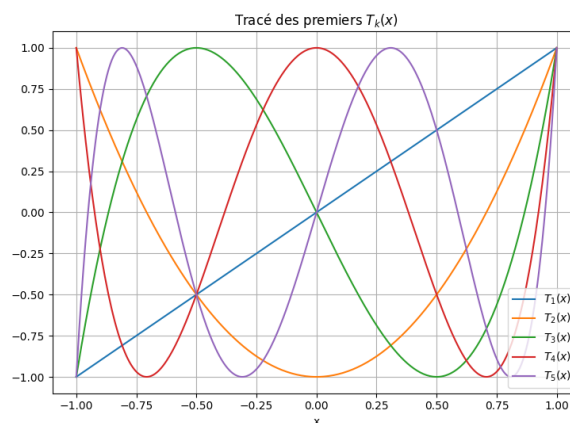


FIGURE 4 – Tracé des polynômes de Tcheb

```

1 x_vals = np.linspace(-1,1,300)
2
3 t1 = fonction(1)
4 t1_vals = t1(x_vals)
5 t2 = fonction(2)
6 t2_vals = t2(x_vals)
7 t3 = fonction(3)
8 t3_vals = t3(x_vals)
9 t4 = fonction(4)
10 t4_vals = t4(x_vals)
11 t5 = fonction(5)
12 t5_vals = t5(x_vals)
13

```

```

14 plt.clf()
15 plt.plot(x_vals,t1_vals, label = r"$T_1(x)$")
16 plt.plot(x_vals,t2_vals, label = r"$T_2(x)$")
17 plt.plot(x_vals,t3_vals, label = r"$T_3(x)$")
18 plt.plot(x_vals,t4_vals, label = r"$T_4(x)$")
19 plt.plot(x_vals,t5_vals, label = r"$T_5(x)$")
20
21 plt.title(r"Trace des premiers $T_k(x)$")
22 plt.xlabel("x")
23 plt.legend()
24 plt.grid(True)
25 plt.show()

```

Pour les racines : soit on compte graphiquement, soit on utilise la fonction `.roots()` qui donne un array avec les racines du polynômes. On retrouve bien la propriété classique des polynômes de Tcheb :

$$\forall n \in \mathbb{N}, \# \text{Rac}(T_n) = n$$

3. On va juste montrer la coïncidence sur $[-1, 1]$ et on pourra conclure (intervalle non réduit à un point) :

```

1 t5_t2_vals = t5(t2(x_vals))
2 t2_t5_vals = t2(t5(x_vals))
3 plt.clf()
4 plt.plot(x_vals,t5_t2_vals, label = r"$T_5 \circ T_2 (x)$")
5 plt.plot(x_vals,t2_t5_vals, label = r"$T_2 \circ T_5 (x)$")
6
7
8 plt.title(r"Vérification que $T_5 \circ T_2 = T_2 \circ T_5$")
9 plt.xlabel("x")
10 plt.legend()
11 plt.grid(True)
12 plt.show()

```

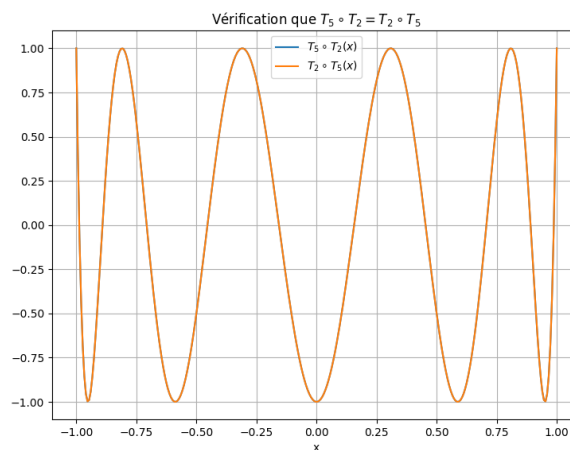


FIGURE 5 – Commutativité de T_2 et T_5

D'où le résultat voulu (ABSL pour les autres).

Exercice 11 On pose P l'ensemble des polynômes à coefficients dans $\{0, 1\}$ tel que leur coefficient constant soit égal à 1. On pose $W = \{z \in \mathbb{C} / \exists Q \in P, \text{ tq } Q(z) = 0\}$. Pour $d \in \mathbb{N}^*$, on pose $P_d = \{Q \in P \text{ tq } \deg(Q) = d\}$ et $W_d = \{z \in \mathbb{C} \text{ tq } \exists Q \in P_d, \text{ tq } Q(z) = 0\}$.

Les questions Python seront renseignées par le symbole [P]. Les questions potentiellement mal retranscrites sont précédées de *.

Question 2. b) [P] Proposer un programme affichant les points de W_d pour $1 \leq d \leq 7$.

c) * [P] Tracer les courbes d'équation $\frac{|z|}{1-|z|} = \frac{|2-z|}{|1-z|}$ et $\frac{|2z-1|}{|z-1|} = \frac{1}{|z|-1}$ sur la même figure. (On peut s'aider de la fonction `plt.contour` présentée dans les documents d'aide Python)

Corrigé 11 b)

```
1 def p(d):
2     res = []
3     for i in range(1<<(d-1), 1<<d): # 1 << k = 2^k (decalage de bits): on parcourt tous les nombres de d-1
4         bits commençant par un 1 (on ajoutera le 1 final apres)
5         res.append(Polynomial([(i & 1<<k) for k in range(d-1)]+[1])) # & est le et logique: on obtient 1 si
6         le k-ieme bit vaut 1, et 0 s'il vaut 0
7     return res
8
9 def w(d):
10    racines = []
11    pd = p(d)
12    for i in range(len(pd)):
13        for elmt in pd[i].roots():
14            racines.append(elmt)
15    return set(racines) # Transforme en ensemble
```

c)

```
1 def f(x,y):
2     return (x**2+y**2)/(1-(x**2+y**2)) - np.abs(2-x-1j*y)/np.abs(1-x-1j*y)
3
4 def g(x,y):
5     return np.abs(2*x+2j*y-1)/np.abs(x+1j*y-1) - 1/((x**2+y**2)-1)
6
7 f = np.vectorize(f)
8 g = np.vectorize(g)
9
10 X = np.arange(-0.99,1,0.001)
11 Y = np.arange(-0.99,1,0.001)
12 X, Y = np.meshgrid(X,Y)
13 Zf = f(X,Y)
14 Zg = g(X,Y)
15 plt.axis("equal") # Pour ne pas avoir des cercles deforms
16 plt.contour(X,Y,Zf,0) # On ne peut pas modifier les couleurs
17 plt.contour(X,Y,Zg,0) # ou labeliser les donnees
18 plt.show()
```

et on obtient le graphique suivant :

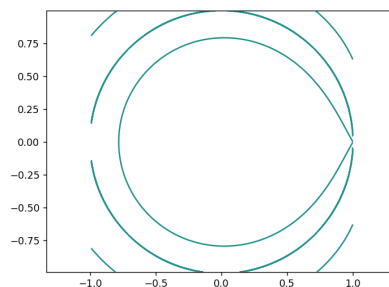


FIGURE 6 – Tracé des courbes d'équation $g : x \mapsto \frac{N(x)}{x^2}$

Exercice 12 On pose : $A_n(\theta) = \sum_{k=1}^n \frac{\sin(k\theta)}{k}$.

1. Écrire un programme Python définissant $A_n(\theta)$ et tracer $A_n(\theta)$ sur $[0, \pi]$ pour $1 \leq n \leq 10$. Que peut-on conjecturer sur le signe de $A_n(\theta)$? Et sur les extremums locaux?

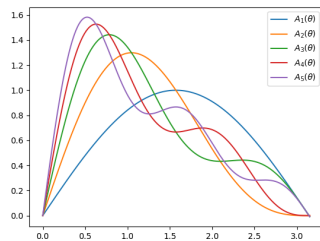
Corrigé 12

1. On a le code et le graphique suivant :

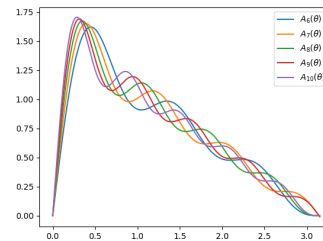
```

1  def a(n,theta) :
2      res = 0
3      for k in range(1,n+1) :
4          res+= np.sin(k * theta) /k
5      return res
6
7
8  x = np.arange(0,np.pi + 0.01, 0.01)
9
10 y = [a(i,x) for i in range(1,11)]
11
12 plt.plot(x,y[0], label=r"$A_1(\theta)$")
13 plt.plot(x,y[1], label=r"$A_2(\theta)$")
14 plt.plot(x,y[2], label=r"$A_3(\theta)$")
15 plt.plot(x,y[3], label=r"$A_4(\theta)$")
16 plt.plot(x,y[4], label=r"$A_5(\theta)$")
17
18
19
20 plt.plot(x,y[5], label=r"$A_6(\theta)$")
21 plt.plot(x,y[6], label=r"$A_7(\theta)$")
22 plt.plot(x,y[7], label=r"$A_8(\theta)$")
23 plt.plot(x,y[8], label=r"$A_9(\theta)$")
24 plt.plot(x,y[9], label=r"$A_{10}(\theta)$")
25
26 plt.legend()
27 plt.show()

```



(a) Tracé des $\theta \mapsto A_i(\theta)$, $i \in \{1, \dots, n\}$



(b) Tracé des $\theta \mapsto A_i(\theta)$ pour $i \in \{1, \dots, n\}$

FIGURE 7 – Le tracé demandé

On peut conjecturer que ces fonctions sont positives et que les extremums locaux sont régulièrement espacés.

4 Notion de norme

Exercice 14 La norme euclidienne sur $\mathbb{R}^2$ est notée $\|\cdot\|$. On pose pour $x \in \mathbb{R}_+$:

$$\begin{cases} C(x) = \{u \in \mathbb{Z}^2 \mid \|u\| \leq x\} \\ C^+(x) = C(x) \cap \mathbb{N}^2 \end{cases}$$

et on note :

$$\begin{cases} N(x) = \#C(x) \\ N^+(x) = \#C^+(x) \end{cases}$$

1. Écrire un programme Python qui calcule $N(x)$ et $N^+(x)$.
2. Vérifier numériquement que $4N^+(x) - N(x) - 4\lfloor x \rfloor$ est constant.
3. Tracer le graphe de $x \mapsto \frac{N(x)}{x^2}$

Corrigé 14

1. On a le code suivant :

```
1 def norm_eucl(z1,z2):
2     return np.sqrt(z1**2 + z2**2)
3
4 def tuple_norme_C(x) :
5     res = []
6     n = int(np.ceil(x)) # Partie entiere superieure
7     for i in range(-n ,n + 1) :
8         for j in range(-n , n+1) :
9             if norm_eucl(i,j) <= x :
10                 res.append((i,j))
11     return res
12
13 def norme_N(x) :
14     return len(tuple_norme_C(x))
15
16 def norme_N_plus(x) :
17     tmp = tuple_norme_C(x)
18     res_to_count = [(i,j) for (i,j) in tmp if (i>=0) & (j >= 0)]
19     return len(res_to_count)
```

2. On vérifie cela sur l'intervalle $[0, 20]$:

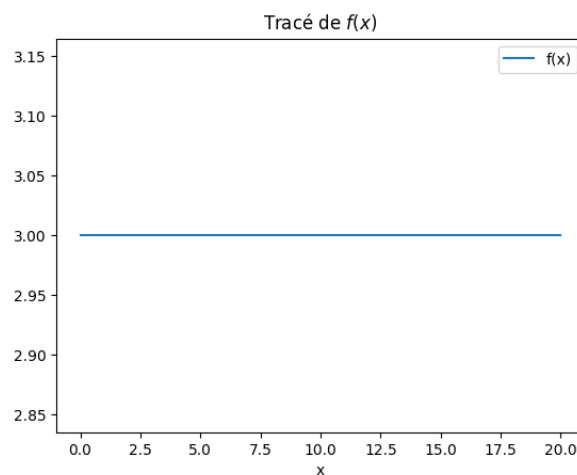


FIGURE 8 – Tracé de $f : x \mapsto 4N^+(x) - N(x) - 4\lfloor x \rfloor$

Attention! Ici la fonction n'est pas vectorisée, donc on ne peut pas directement appliquer f à un `np.array`. Pour cela, on utilisera `f_vect = np.vectorize(f)`.

3. Il ne reste plus qu'à définir et tracer cette fonction, cette fois-ci sur $[1, 20]$ pour ne pas diviser par 0 :

```

1 def g(x) :
2     return norme_N(x)/(x**2)
3
4 t_vals = np.linspace(1,50,300)
5 #f_vect = np.vectorize(f)
6 #f_vals = f_vect(t_vals)
7
8 g_vect = np.vectorize(g)
9 g_vals = g_vect(t_vals)
10
11 plt.clf()
12 #plt.plot(t_vals,f_vals,label = r"f(x)")
13 plt.plot(t_vals,g_vals,label = r"g(x)")
14 plt.title(r"Trace de $g(x)$")
15 plt.grid(True)
16 plt.xlabel("x")
17 plt.legend()
18 plt.show()

```

et on obtient le graphique suivant :

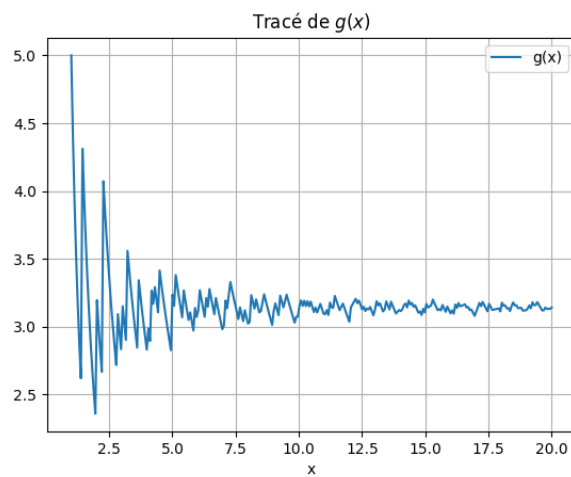


FIGURE 9 – Tracé de $g : x \mapsto \frac{N(x)}{x^2}$

Exercice 16 Si $A \in \mathcal{M}_n()$, on note $\exp(A) = \sum_{k=0}^{+\infty} \frac{A^k}{k!}$.

1. Démontrer l'existence de $\exp(A)$.
2. Pour $t \in \mathbb{R}$, on pose $A(t) = \begin{pmatrix} 0 & -t \\ t & 0 \end{pmatrix}$ et $B(t) = \exp(A(t)) = \begin{pmatrix} a(t) & b(t) \\ c(t) & d(t) \end{pmatrix}$.
 - (a) A l'aide de PYTHON, tracer les fonctions a, b, c, d sur $[0, 2\pi]$ puis conjecturer leur expression.

Corrigé 16

1. Norme triple, la classique convergence absolue en dimension finie.
2. (a) On a le code et le graphique suivant :

```

1  def exp_A(n) :
2      def aux(t) :
3          mat_a = np.array([[0,-t],[t,0]])
4          tmp = np.array([[1,-t],[t,1]]) # terme en k=0 et k=1
5          res = tmp.copy()
6          ind = 1
7          while ind < n :
8              ind +=1
9              tmp = np.dot(tmp,mat_a) * (1/ind)
10             res += tmp
11         return res
12     return aux
13
14 N = 500
15 t_vals = np.arange(0, 2* np.pi, 0.01)
16 fonctions = exp_A(N)
17 mat_vals = np.array([fonctions(t) for t in t_vals])
18
19 vals_a = mat_vals[:,0,0]
20 vals_b = mat_vals[:,0,1]
21 vals_c = mat_vals[:,1,0]
22 vals_d = mat_vals[:,1,1]
23
24 plt.plot(t_vals,vals_a,label=r"a(t)")
25 plt.plot(t_vals,vals_b,label=r"b(t)")
26 plt.plot(t_vals,vals_c,label=r"c(t)")
27 plt.plot(t_vals,vals_d,label=r"d(t)")
28 plt.grid(True)
29 plt.legend()
30 plt.show()

```



FIGURE 10 – Tracé des 4 fonctions.

5 Espaces probabilisés

Exercice 25 Deux joueurs A et B s'affrontent lors d'un match de badminton selon les règles suivantes :

- A commence à servir
- le joueur qui gagne le point sert pour le point suivant
- le joueur qui sert a probabilité $2/3$ de gagner le point et $1/3$ de le perdre

Pour $n \in \mathbb{N}^*$, on note A_n (resp. B_n) l'événement « A (resp. B) gagne le n-ème point » ainsi que $a_n = P(A_n)$ et $b_n = P(B_n)$.

5. Grâce à python, représenter graphiquement les dix premiers termes des suites (a_n) et (b_n) .

Corrigé 25

On a prouvé dans la question 3. que $\begin{pmatrix} a_{n+1} \\ b_{n+1} \end{pmatrix} = M \begin{pmatrix} a_n \\ b_n \end{pmatrix}$ avec $M = \begin{pmatrix} 2/3 & 1/3 \\ 1/3 & 2/3 \end{pmatrix}$

```
1 # Calcul des valeurs
2 M = np.array([[2/3, 1/3],[1/3, 2/3]])
3 # a1 sera present dans a[0]
4 a = np.zeros(10)
5 b = np.zeros(10)
6 a[0] = 2/3
7 b[0] = 1/3
8 for i in range(9): # on veut generer 9 valeurs car a1 et b1 sont deja connus
9     etat_actuel = np.array([a[i], b[i]])
10    etat_suivant = np.dot(M,etat_actuel)
11    a[i+1] = etat_suivant[0]
12    b[i+1] = etat_suivant[1]
13
14 # Affichage des valeurs
15 x = np.array([i for i in range(1,11)])
16 plt.plot(x,a,color='r',linestyle=':',marker='o',label="(an)")
17 plt.plot(x,b,color='b',linestyle=':',marker='o',label="(bn)")
18 plt.title("10 premiers termes des suites (an) et (bn)")
19 plt.legend()
20 plt.show()
```

et on obtient le graphique suivant :

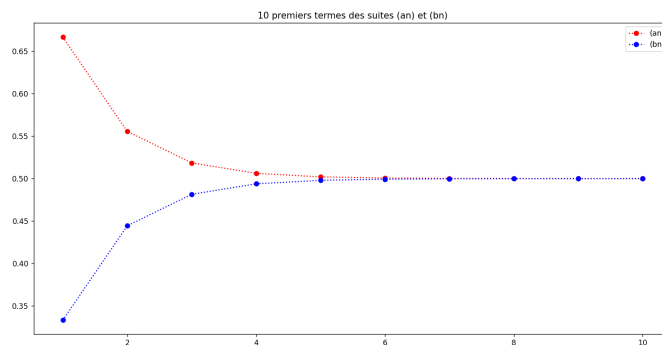


FIGURE 11 – 10 premières valeurs de (a_n) et (b_n)

Exercice 26 On note S_n^k le nombre de surjections existant entre un ensemble à n éléments et un ensemble à k éléments.

5. Implémenter un programme python permettant de calculer les S_n^k .

Corrigé 26

On a prouvé dans la question 4. que $S_n^k = k \left(S_{n-1}^k + S_{n-1}^{k-1} \right)$. On en déduit ainsi le code suivant :

```
1 def s(n,k):
2     res = np.zeros((n+1,k+1),dtype=int) # Les elements d'indice 0 ne seront pas utilises
3     # Conditions initiales d'apres la question 1
4     for i in range(n+1):
5         res[i,1] = 1
6     for i in range(k+1):
7         res[i,i] = np.math.factorial(i)
8     # Relation de recurrence
9     for i in range(n+1):
10        for j in range(min(k+1,i)): # On s'arrete a la diagonale car au-dela, il n'y a pas de surjection qui
            convient
11            res[i,j] = j * (res[i-1,j] + res[i-1,j-1])
12    return res[n,k]
```

6 Espaces euclidiens

Exercice 27 On considère $E = \mathbb{R}^3$ muni de la norme euclidienne. Soit $q : E \rightarrow \mathbb{R}$ telle que : $\forall u = (x, y, z) \in E, q(u) = x^2 + y^2 - z^2$. Soit $C = \{u \in E \mid q(u) = 0\}$ et $D = \{u \in \mathbb{C} \mid \|u\| = 1\}$.

1) Tracer la surface C avec Python. De quel type de surface s'agit-il?

4) Représenter D à l'aide de Python.

Corrigé 27 1)

```
1 ax = Axes3D(plt.figure())
2
3 def f(r, theta):
4     # En cartésienne, f = x^2 + y^2 donc vaut r en cylindrique
5     return r
6
7 r = np.arange(0, 1, 0.02)
8 theta = np.arange(0, 2*np.pi, 0.02)
9 R, Theta = np.meshgrid(r, theta) # R est une matrice ou chaque ligne vaut r et
10                                     # Theta est une matrice ou chaque colonne vaut theta
11 X = R * np.cos(Theta)
12 Y = R * np.sin(Theta)
13 Zpos = f(R, Theta)
14 Zneg = -f(R, Theta)
15 ax.plot_surface(X, Y, Zpos, color="blue")
16 ax.plot_surface(X, Y, Zneg, color="blue")
17 plt.show()
```

La surface obtenue est un double cône (voir ci-dessous)

4)

```
1 def x(t):
2     return np.cos(t)
3 def y(t):
4     return np.sin(t)
5 T = np.arange(0, 2*np.pi, 0.01)
6 X = x(T)
7 Y = y(T)
8 plt.axis('equal') # Pour que le cercle ne soit pas déformé
9 plt.plot(X, Y)
10 plt.grid()
11 plt.show()
```

Les deux figures obtenues sont :

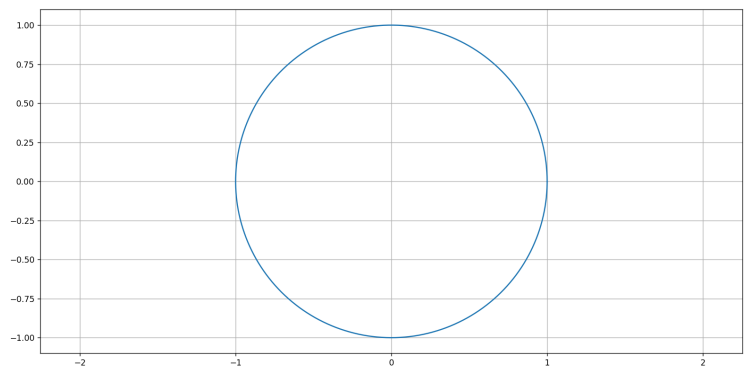
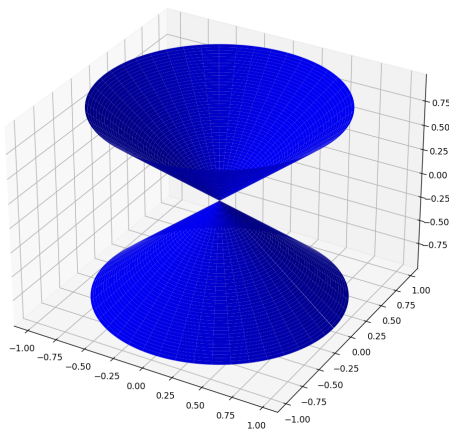


FIGURE 12 – Tracés des surfaces C (à gauche) et D (à droite)

Exercice 28 Soit $(F_n) \in \mathbb{N}^{\mathbb{N}}$ telle que $F_0 = 0$, $F_1 = 1$ puis $F_{n+2} = F_{n+1} + F_n$. Soit $M \in S_n(\mathbb{R})$.

Pour $n \geq 2$, on définit $A_n = (F_{i+j-2})_{1 \leq i, j \leq n} \in M_n(\mathbb{R})$.

b) Écrire une fonction python qui prend n en argument et renvoie A_n sous forme d'un tableau numpy.

c) Écrire une fonction python qui détermine les valeurs propres de A_n .

Corrigé 28 b)

```
1 def a(n):
2     # On genere les n premiers termes de F
3     fibonacci = np.zeros(2*n, dtype=int)
4     fibonacci[0] = 0
5     fibonacci[1] = 1
6     for i in range(2, 2*n):
7         fibonacci[i] = fibonacci[i-1] + fibonacci[i-2]
8     # On genere la matrice A
9     res = np.zeros((n,n), dtype=int)
10    for i in range(n):
11        for j in range(n):
12            res[i,j] = fibonacci[i+j]
13    return res
```

c)

```
1 # Il existe une fonction, appelee dans le document Matrice, qui le fait:
2 alg.eigvals(a(n))
3
4 #Sinon, via les racines du polynome caracteristique:
5 np.roots(np.poly(a(n)))
```

Exercice 29 Soit $A \in M_n(\mathbb{R})$.

c) Utilisation de python

1. Ecrire une fonction `randO(n)` qui génère une matrice aléatoire A et renvoie la matrice orthogonale O de la question précédente (telle que $A = OT$ avec T triangulaire supérieure).
2. Ecrire une fonction `N1`, de la variable matricielle A , qui renvoie $N_1(A)$. On pourra utiliser les fonction `numpy.sum` et `numpy.abs`.
3. Ecrire une fonction `test(n)` qui, sur 1000 tests, renvoie le minimum et le maximum des valeurs de N_1 , pour des matrices orthogonales aléatoires.

Corrigé 29

1. En utilisant la fonction donnée dans l'énoncé, on a

```
1 def randO(n):
2     a = np.zeros((n,n),dtype=int)
3     for i in range(n):
4         for j in range(n):
5             a[i,j] = rd.randint(-100,100)
6     return np.linalg.qr(a)[0]
```

2. En utilisant les deux fonctions suggérées, on a

```
1 def n1(a):
2     return np.sum(np.abs(a))
```

3. En utilisant les fonctions écrites préalablement, on a

```
1 def test(n):
2     minimum = n1(randO(n))
3     maximum = minimum
4     for _ in range(999):
5         valeur = n1(randO(n))
6         if valeur > maximum:
7             maximum = valeur
8         if valeur < minimum:
9             minimum = valeur
10    return minimum,maximum
```

Exercice 30 NON FAIT

Exercice 31 Soit S la sphère unité de R_n . Pour $M \in S_n(\mathbb{R})$, on note $\lambda_1(M), \dots, \lambda_n(M)$ ses valeurs propres triées dans l'ordre croissant. Pour un sous-espace vectoriel F non nul de E , on note $R_M(F) = \max_{X \in F \cap S} X^T M X$.

1. Écrire une fonction qui, pour $M \in S_n(\mathbb{R})$, renvoie la liste des valeurs propres triées par ordre croissant : $[\lambda_1(M), \dots, \lambda_n(M)]$.
2. Écrire une fonction qui, pour $(A, B) \in S_n(\mathbb{R})^2$ avec $A \neq B$, renvoie la liste des $\frac{|\lambda_k(A) - \lambda_k(B)|}{\max\{-\lambda_1(A-B), \lambda_n(A-B)\}}$ pour $k \in \llbracket 1, n \rrbracket$.
3. Tester cette fonction pour des matrices symétriques aléatoirement choisies.

Corrigé 31

1. La fonction `alg.eigvals` renvoie les valeurs propres d'une matrice

```
1 def valeurs_propres(m):
2     return np.sort(alg.eigvals(m))
```

2. En utilisant la fonction précédente

```
1 def liste(a,b):
2     valeurs_propres_a = valeurs_propres(a)
3     valeurs_propres_b = valeurs_propres(b)
4     valeurs_propres_a_b = valeurs_propres(a-b)
5     return np.abs(valeurs_propres_a - valeurs_propres_b) / max(-valeurs_propres_a_b[0],
        valeurs_propres_a_b[-1])
```

3. Il suffit de générer des matrices symétriques aléatoirement

```
1 def test(n):
2     a = np.zeros((n,n),dtype=int)
3     b = np.zeros((n,n),dtype=int)
4     for i in range(n):
5         for j in range(i,n):
6             a[i,j] = rd.randint(-100,100)
7             b[i,j] = rd.randint(-100,100)
8             a[j,i] = rd.randint(-100,100)
9             b[j,i] = rd.randint(-100,100)
10    return liste(a,b)
```

7 Suites et séries de fonctions

Exercice 32 Dans l'espace $E = C^0([0, 1], \mathbb{R})$, muni de la norme de la convergence uniforme, on considère $E_{0,1} = \{f \in E / f(0) = 0, f(1) = 1\}$. Pour $f \in E_{0,1}$ on pose $T(f) : [0, 1] \rightarrow \mathbb{R}$ où $T(f)(x) = \frac{f(3x)}{2}$ si $x \in [0, \frac{1}{3}]$, $T(f)(x) = \frac{1}{2}$ si $x \in [\frac{1}{3}, \frac{2}{3}]$ et $T(f)(x) = \frac{f(3x-2)+1}{2}$ si $x \in [\frac{2}{3}, 1]$.

- a) Ecrire une fonction python qui a $f \in E_{0,1}$ et x associe $T(f)(x)$. Tracer les graph de Id et $T(Id)$.
e) Ecrire une fonction récursive qui retourne $T^n(f)$. Tracer le graph de $T^5(Id)$.

Corrigé 32 a)

```

1 def t(f):
2     def fct(x):
3         if 0 <= x < 1/3:
4             return f(3*x)/2
5         if x < 2/3:
6             return 1/2
7         return (f(3*x-2)+1)/2
8     return fct
9
10 f = np.vectorize(t(lambda x: x))
11 X = np.arange(0, 1.01, 0.01)
12 Y = f(X)
13 plt.plot(X, Y, color="orange", label="T(Id)")
14 plt.plot(X, X, color="blue", label="Id")
15 plt.legend()
16 plt.show()

```

et on obtient le graphe suivant :

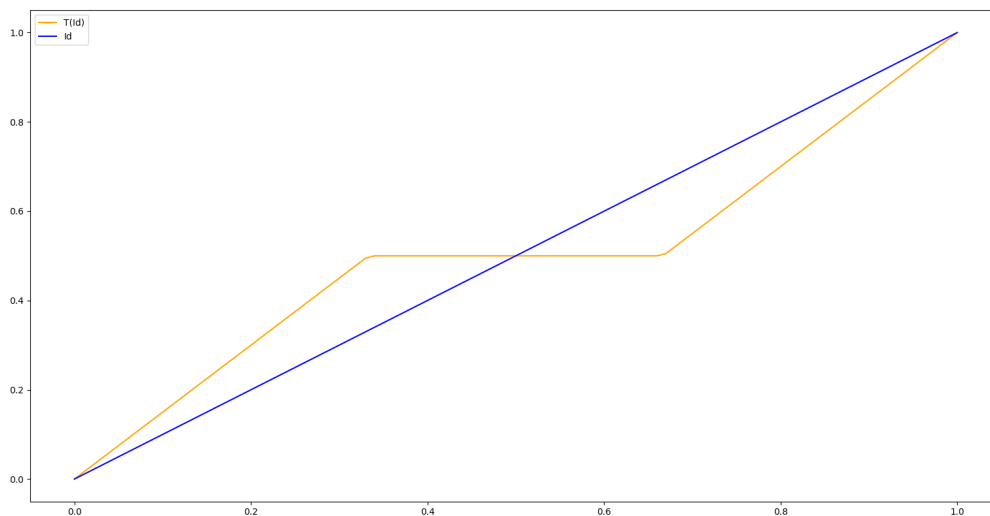


FIGURE 13 – Tracé de $I_n(x)$ et $T(I_n)(x)$

e)

```

1 def t_puissance(n, f):
2     if n==0:
3         return f
4     return t(t_puissance(n-1, f))
5
6 f = np.vectorize(t_puissance(5, lambda x: x))
7 X = np.arange(0, 1.01, 0.01)
8 Y = f(X)
9 plt.plot(X, Y, label="T^5(Id)")
10 plt.legend()
11 plt.show()

```

et on obtient le graphe suivant :

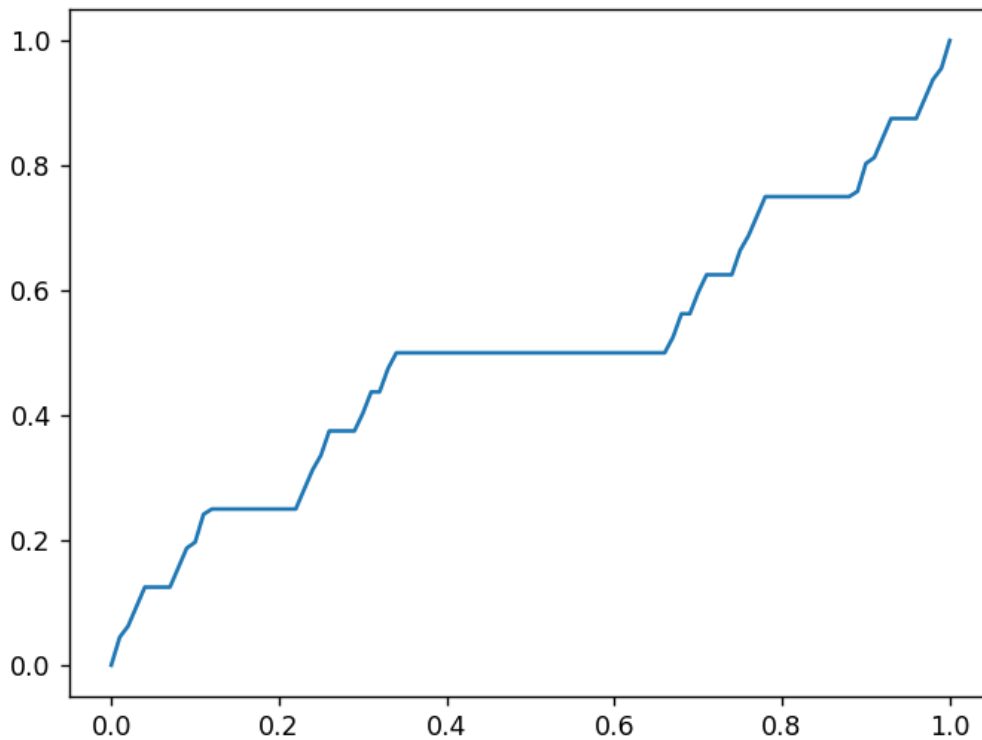


FIGURE 14 – Tracé de $T^5(I_n)(x)$

Exercice 33 Pour $n \in \mathbb{N}$ et $x \in \mathbb{R}_+$, soient $P_n(x) = \sum_{k=0}^n \frac{(nx)^k}{k!}$ et $f_n(x) = e^{-nx} P_n(x)$.

- Ecrire une fonction qui renvoie la valeur de $P_n(x)$. Vérifier que $P_7(0.1) = 2.0137348...$
- Tracer le graph de la restriction de f_n à $[0, 2]$ pour quelques valeurs de n . Conjecturer un résultat relatif à la convergence de la suite de fonctions (f_n) .

Corrigé 33 a)

```

1 def p(n,x):
2     terme = 1
3     somme = terme
4     for i in range(1,n+1):
5         terme *= (n*x)/i
6         somme += terme
7     return somme
8
9 print(p(7,0.1))

```

b)

```

1 X = np.arange(0,2.01,0.01)
2 for i in range(1,22,2):
3     plt.plot(X,p(i,X),label="n = "+str(i))
4 plt.show()

```

et on obtient le graphe suivant :

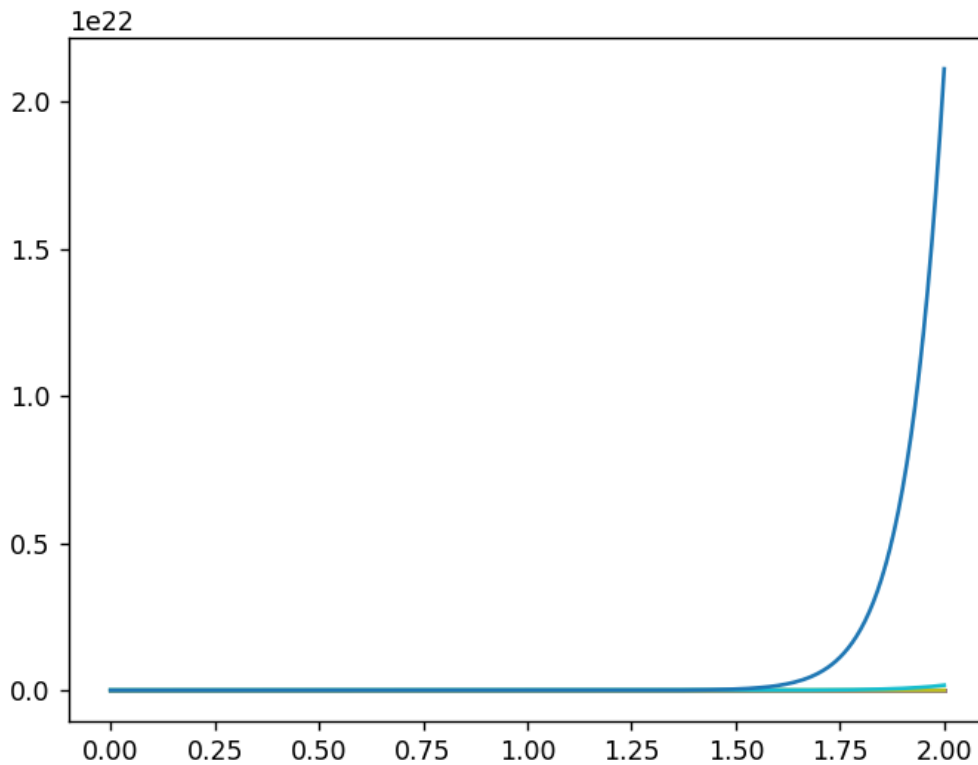


FIGURE 15 – Tracé de f_n sur $[0, 2]$

Exercice 34 On note b_n le nombre de 1 dans la représentation binaire de n .

On pose $\varepsilon_n = (-1)^{b_n}$

1. Coder une fonction `epsilon(n)` qui renvoie les valeurs de ε_k jusqu'à n
2. Montrer avec Python que $\forall n \in \mathbb{N} \ \varepsilon_{2n} = \varepsilon_n$ et $\varepsilon_{2n+1} = -\varepsilon_n$ puis démontrer ce résultat
3. En déduire une fonction `epsilon2(n)` utilisant les relations précédentes

On pose $S_n = \sum_{k=0}^n \varepsilon_k$

4. Conjecturer avec Python les valeurs de S_n et démontrer ce résultat

Corrigé 34 1)

```
1 def nb_un(k):
2     return np.sum(np.array([(k>>i & 1) for i in range(int(np.floor(np.log(k)/np.log(2))+1))])) # Realise l'
        operation: decale les bits de k de i rangs puis realise le "et" logique avec 1, donc au total, vaut
        1 si et seulement si le i-ieme bit de k vaut 1, et 0 sinon
3
4 def epsilon(n):
5     return np.array([(-1)**(nb_un(k)) for k in range(1,n+1)])
```

2)

```
1 for n in range(1000):
2     print("Pour n =",n,", on a epsilon 2n =",epsilon(2*n),", epsilon n =",epsilon(n),"et epsilon 2n+1 =",
        epsilon(2*n+1))
```

3)

```
1 def epsilon2(n):
2     if n==0:
3         return 1 # 0 n'a pas de 1 dans sa representation binaire
4     if n%2==0:
5         return epsilon2(n//2)
6     return -1*epsilon2((n-1)//2)
```

4)

```
1 def s(n):  
2     return np.sum(np.array([epsilon2(k) for k in range(n+1)]))  
3  
4 s = np.vectorize(s)  
5  
6 X = range(0,100)  
7 Y = s(X)  
8 plt.scatter(X,Y)  
9 plt.show()
```

et on obtient le graphique suivant :

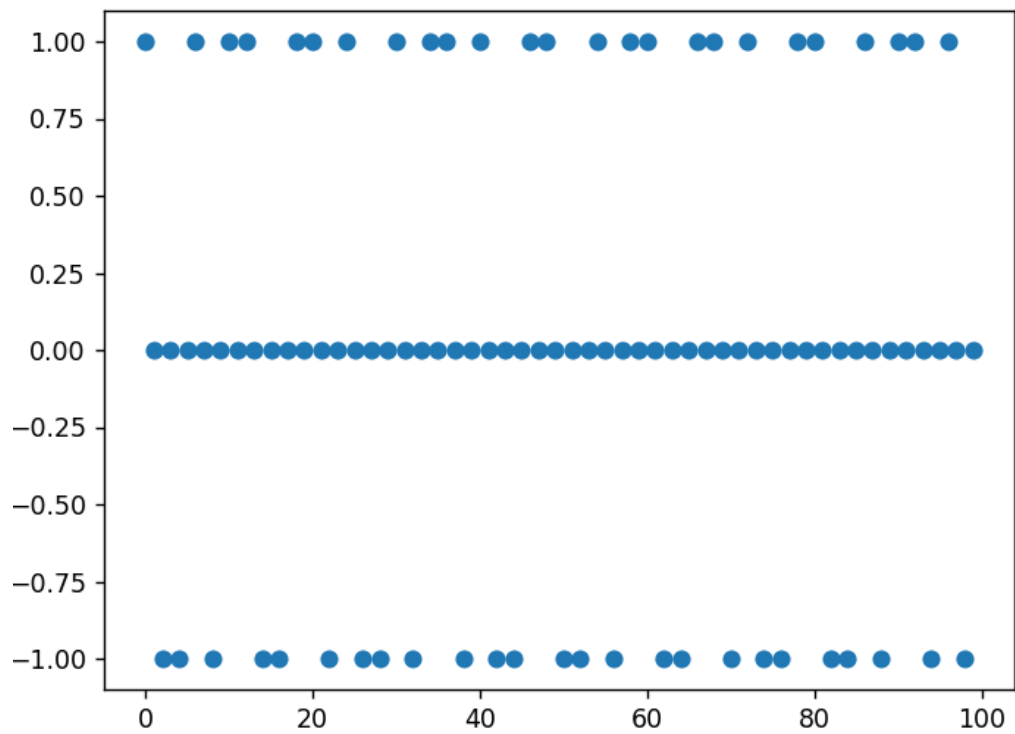


FIGURE 16 – Tracé de S_n

Exercice 35 On pose l'équation différentielle $y'' - xy = 0$ (E).

$$a_n = \prod_{i=1}^n (i - 1/3) \text{ et } a_0 = 1$$

On considèrera également la série entière $\sum_{n=0}^{+\infty} \frac{x^{3n}}{9^n n! a_n}$.

2) Coder un programme calculant a_n .

3) Coder un programme calculant une approximation de la série entière - L'énoncé précise que l'on néglige les termes inférieurs à 0.05 et que l'on ne doit pas utiliser la fonction factorielle pour le calcul.

4) Faire apparaitre sur un même graphique les valeurs prises par f et l'approximation de la série entière sur $[0, 3]$. Conjecturer.

Corrigé 35 2)

```
1 def a(n):
2     return np.prod(np.array([(k-1/3) for k in range(1,n+1)]))
```

3)

```
1 def serie_entiere(x):
2     res = 0
3     xcube = x**3
4     xcube_actuel = 1
5     neuf_actuel = 1
6     factoriel_actuel = 1
7     n = 0
8     terme = xcube_actuel / (neuf_actuel * factoriel_actuel * a(n))
9     while terme > 0.05:
10         res += terme
11         xcube_actuel *= xcube
12         neuf_actuel *= 9
13         n+=1
14         factoriel_actuel *= n
15         terme = xcube_actuel / (neuf_actuel * factoriel_actuel * a(n))
16     return res
```

4)

```
1 def f(x,t):
2     return np.array([x[1],x[0]*t])
3
4 serie_entiere = np.vectorize(serie_entiere)
5 X = np.arange(0,10.01,0.01)
6 Y = serie_entiere(X)
7 Z = integr.odeint(f,np.array([1.,0.],dtype=float),X)
8 plt.plot(X,Y)
9 plt.plot(X,Z)
10 plt.show()
```

et on obtient le graphique suivant :

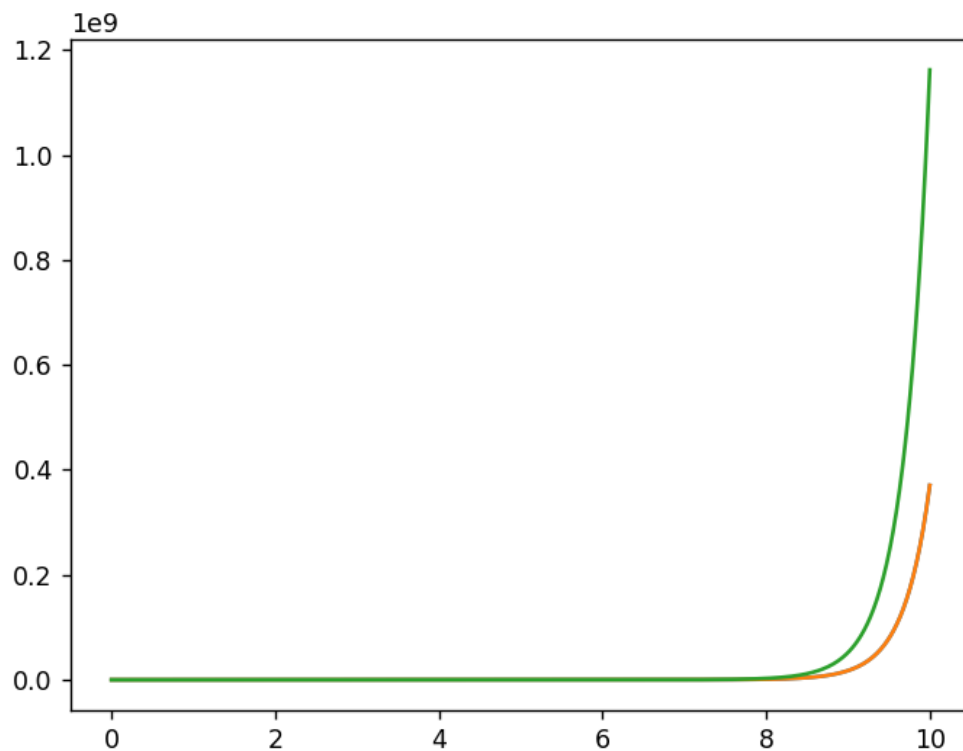


FIGURE 17 – Tracé de l'approximation de la série entière et de la vraie fonction

Exercice 36 1. Question Python

Soit $p \in \mathbb{N}$, calculer la valeur de $(1-x)^{p+1} \sum_{n=0}^{+\infty} n^p x^n$ quand $x \rightarrow 1^-$. Conjecturer la limite en 1^- .

```

1 def serie(p,x,N):
2     return (1-x)**(p-1) * np.sum(np.array([(n**p * x**n) for n in range(N)]))
3
4 X = np.arange(0,1,0.01)
5 for p in range(10):
6     plt.plot(X,serie(p,X,10000),label="p="+str(p))
7 plt.show()

```

et on obtient le graphique suivant :

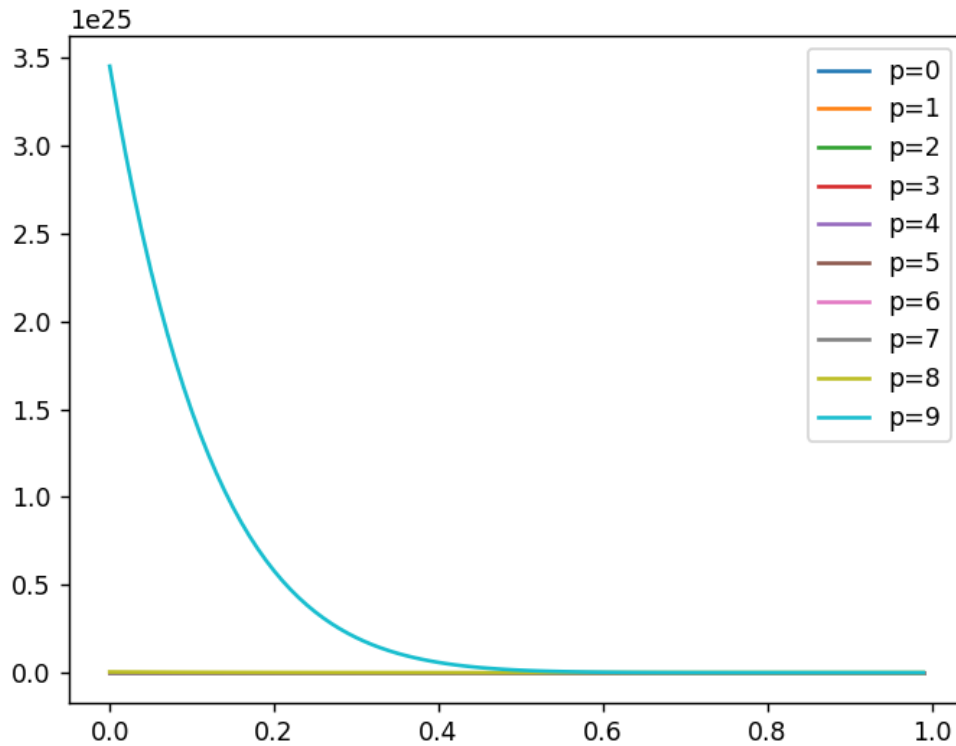


FIGURE 18 – Tracé de $(1-x)^{p+1} \sum_{n=0}^{+\infty} n^p x^n$ quand $x \rightarrow 1^-$

Exercice 37 Enoncé à ajouter, graphes et tests à faire, valeur C à lire

Corrigé 37 2a)

```

1 def u(n):
2     return lambda x: np.exp(x) - np.sum(np.array([x**k / np.math.factorial(k) for k in range(n+1)]))
3
4 def somme_partielle(x,N):
5     return np.sum(np.array([(-1)**n * n * u(2*n)(x)]))
6
7 somme_partielle = np.vectorize(somme_partielle)
8 X = np.arange(-0.99,1,0.01)
9 for N in range(1,102,10):
10     plt.plot(X,somme_partielle(X,N),label="N="+str(N))
11 plt.show()

```

2b)

```

1 X = np.arange(-0.99,1,0.01)
2 for N in range(1,102,10):
3     plt.plot(X,somme_partielle(X,N)/((x+1)*np.cos(x) + x*np.sin(x)-np.exp(x)),label="N="+str(N))
4 plt.show()

```

8 Intégration

Exercice 40 Soit $U : \mathbb{N}^2 \rightarrow \mathbb{N}$ telle que :

$$U(0, 0) = 1.$$

$$\forall n \in \mathbb{N}^*, U(0, n) = U(n, 0) = 0.$$

$$\forall n, m \in \mathbb{N}^*, U(n, m) = U(n-1, m-1) + m U(n-1, m).$$

1. a) Écrire une fonction Python qui renvoie $U(n, m)$. Calculer $U(6, 4)$.

c) Soit pour $n \in \mathbb{N}^*$, $P_n = X(X-1) \dots (X-n+1)$ et $P_0 = 1$. Calculer $\sum_{k=0}^n U(n, k) P_k$ pour $n \in \llbracket 0, 10 \rrbracket$.

Corrigé 40 1. a)

```
1 def u(n,m):
2     if (n,m)==(0,0):
3         return 1
4     if n==0 or m==0:
5         return 0
6     return u(n-1,m-1)+m*u(n-1,m)
```

c)

```
1 def somme(n):
2     cmpt = Polynomial([0])
3     polynome = Polynomial([1])
4     for k in range(n + 1):
5         cmpt += u(n, k) * polynome
6         polynome = polynome * Polynomial([-k,1])
7     return cmpt
8
9 for n in range(1,11):
10     print("n =",n,":",somme(n))
```

et on peut conjecturer que $\text{somme}(i) = X^i$ pour tout $i \in \mathbb{N}$.

9 Espaces vectoriels normés, topologie

Exercice 41 Enoncé à ajouter, graphes et tests à faire

Corrigé 41 a)

```
1 def k(x,y):
2     if x<=y:
3         return np.sin(x)*np.cos(y)
4     return np.cos(x)*np.sin(y)
5
6 def l(f,x,nb_points):
7     ecart = np.pi/(2*nb_points)
8     return np.sum(np.array([ecart * k(x,y) * f(y) for y in np.arange(0,2*np.pi + ecart,ecart)]))
```

b)

```
1 def f(n,x):
2     return np.sin(2*n*x)
3
4 def g(n,x):
5     return (4*n*n - 1) * l(lambda x: np.sin(2*n*x))(x)
6
7 X = np.arange(0,np.pi/2+0.01,0.01)
8 for n in [1,2,5,10]:
9     plt.plot(X,f(n,X),label="n="+str(n),linestyle=":")
10    plt.plot(X,g(n,X),label="n="+str(n),linestyle="--")
11 plt.show()
```

Exercice 42 Enoncé à ajouter, graphes et tests à faire

Corrigé 42 a)

```
1 def r(theta):
2     mat = np.eye(3)
3     mat[1,1] = np.cos(theta)
4     mat[2,2] = np.cos(theta)
5     mat[1,2] = -1*np.sin(theta)
6     mat[2,1] = np.sin(theta)
7     return mat
```

b)

```
1 def alea():
2     res = np.ones(3)
3     res[0] = rd.random() * np.exp(1j * np.random * 2 * np.pi) # On genere un complexe aleatoire de module
4     # inferieur a 1
5     res[1] = (rd.random() * (1 - np.abs(res[0])**2) * np.exp(1j * np.random * 2 * np.pi) # On genere un
6     # complexe aleatoire tel que la somme des carres des deux modules soit inferieure a 1
7     res[2] = (1 - np.abs(res[0])**2 - np.abs(res[1])**2) * np.exp(1j * np.random * 2 * np.pi) # On genere un
8     # complexe aleatoire tel que la somme des carres des modules fasse 1
9     return res
```

c)

```
1 def fct():
2     res = np.ones(10000)
3     for i in range(10000):
4         z = alea()
5         theta = rd.random() * 2 * np.pi
6         res[i] = np.dot(np.transpose(z), np.dot(r(theta), z))
7     return res
```

Exercice 43 Enoncé à ajouter, tests à faire

Corrigé 43 a)

```
1 def h(n):
2     return np.array([[1/(i+j+1) for j in range(n)] for i in range(n)]) # +1 car on a i = i-1 et j=j-1
3
4 for n in range(2,6):
5     print("Les valeurs propres de H",n,"sont:",alg.eigvals(h(n)))
6
7 def q(n,x):
8     return np.vdot(np.dot(h(n),x),x)
```

Exercice 44 Enoncé à ajouter, graphes et tests à faire

Corrigé 44 a)

```
1 def r(theta):
2     mat = np.eye(3)
3     mat[1,1] = np.cos(theta)
4     mat[2,2] = np.cos(theta)
5     mat[1,2] = -1*np.sin(theta)
6     mat[2,1] = np.sin(theta)
7     return mat
```

b)

```
1 def alea():
2     res = np.ones(3)
3     res[0] = rd.random() * np.exp(1j * np.random * 2 * np.pi) # On genere un complexe aleatoire de module
4     # inferieur a 1
5     res[1] = (rd.random() * (1 - np.abs(res[0])**2) * np.exp(1j * np.random * 2 * np.pi) # On genere un
6     # complexe aleatoire tel que la somme des carres des deux modules soit inferieure a 1
7     res[2] = (1 - np.abs(res[0])**2 - np.abs(res[1])**2) * np.exp(1j * np.random * 2 * np.pi) # On genere un
8     # complexe aleatoire tel que la somme des carres des modules fasse 1
9     return res
10
11 res = np.ones(1000)
12 theta = np.linspace(0,2*np.pi+0.01,1000)
13 for i in range(1000):
14     z = alea()
15     res[i] = np.dot(np.transpose(z),np.dot(r(theta[i])),z)
16 plt.scatter(theta,res)
17 plt.show()
```

Exercice 45 Enoncé à ajouter, graphes et tests à faire

Corrigé 45 a)

```
1 def val_propre_max(m):  
2     return np.amax(alg.eigvals(m))
```

b)

```
1 n = 10 # Taille des matrices  
2 for _ in range(10):  
3     print(val_propre_max(np.array([(1+rd.random()) for _ in range(n*n)]).reshape((n,n))))
```

10 Variables aléatoires

Exercice 46 Fait par Pierre, il faut que je l'ajoute

Corrigé 46 1)

```
1 def passage(n):  
2     return rd.randint(1,n)
```

2)

```
1 def pythonx(n):  
2     nb_min = 1  
3     while n > 1:  
4         n -= passage(n)  
5         nb_min += 1  
6     return nb_min
```

4)

```
1 def e(s,n):  
2     return np.sum(np.array([pythonx(n) for _ in range(s)]))/s  
3  
4 e = np.vectorize(e)  
5 N = np.arange(0,50,1)  
6 Y = e(500,N)  
7 plt.plot(N,Y)  
8 plt.show()
```

et on obtient le graphique suivant :

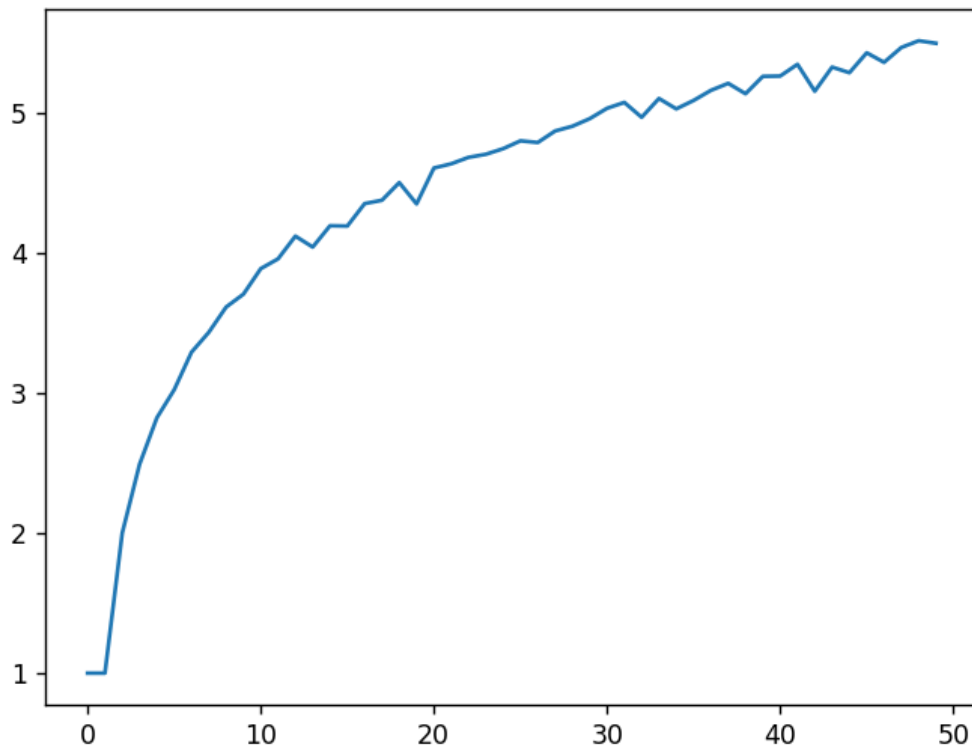


FIGURE 19 – Tracé de l'espérance de X_n

Exercice 47 Fait par Pierre, il faut que je l'ajoute

Corrigé 47 1.

```
1 a = np.array([[1,1,0,1],[1,0,1,1],[0,1,1,0],[1,1,0,1]])
2 valpropres, vecteurspropres = alg.eig(a)
```

2.

```
1 def norme(m):
2     return np.amax(m)
3
4 def w(k):
5     if k==0:
6         return rd.random(4,1)
7     return 1/norme(a)*np.dot(a,log(w(k-1)))
```

Exercice 48 Fait par Pierre, il faut que je l'ajoute

Corrigé 48 1) a)

```
1 def x(k,n,p):  
2     return np.sum(np.array([(rd.binomial(1,p)*2-1) for _ in range(k)])) % n
```

b)

```
1 valeurs = np.array([x(100,21,1/2) for _ in range(200)])  
2 plt.hist(valeurs)  
3 plt.show()
```

c)

```
1 def a(n,p):  
2     mat = np.zeros((n,n))  
3     for i in range(n):  
4         mat[(i+1)%n,i] = p  
5         mat[(i-1)%n,i] = 1-p  
6     return mat
```

2) a)

```
1 def u(k, n, p):  
2     if k == 0:  
3         return np.array([1]+[0 for _ in range(n-1)])  
4     return np.dot(alg.matrix_power(a(n,p),k),u(0,n,p))
```

Exercice 49 Enoncé à ajouter, graphes et tests à faire

```
1 def k(a,n,x,e):
2     return np.sum(np.array([(x()-e)/k**a for k in range(1,n+1)]))
3
4 lambda = 2 # Valeur arbitraire
5 k = np.vectorize(k)
6 N = range(1,100)
7 for a in range(1,10):
8     plt.scatter(N,k(a,N,np.binomial(1,1/2)*2-1,0),label="Rademacher: a="+str(a),linestyle="-") # np.binomial
9     # (1,1/2) represente une experience de Bernoulli de parametre 1/2
10    plt.scatter(N,k(a,N,np.poisson(lambda),lambda),label="Poisson de lambda="+str(lambda)+" : a="+str(a),
11                linestyle="--")
12 plt.show()
```

Exercice 50 Fait par Pierre, il faut que je l'ajoute

Corrigé 50 a)

```
1 def x(n):  
2     return rd.poisson(n)  
3  
4 def z(n):  
5     if z==1:  
6         return x(1)  
7     return np.sum(np.array([x(n) for _ in range(z(n-1))]))
```

Exercice 51 Enoncé à ajouter, graphes et tests à faire

Corrigé 51 a)

```
1 def marche_aleatoire(n):
2     return np.sum(np.array([np.binomial(1,1/2)*2-1 for _ in range(n)]))
```

b)

```
1 def temps_retour():
2     cmpt = np.binomial(1,1/2)*2-1
3     indice = 1
4     while cmpt != 0:
5         cmpt += np.binomial(1,1/2)*2-1
6         indice += 1
7     return indice
```

c)

```
1 def binom(k,n):
2     return np.math.factorial(n) // (np.math.factorial(k) * np.math.factorial(n-k))
3
4 def proba_t(n):
5     cmpt = 0
6     for _ in range(10000):
7         if marche_aleatoire(n)==0:
8             cmpt += 1
9     return cmpt/10000
10
11 def alpha(n):
12     return proba_t(2*n) * 2 * 4**n / binom(n,2*n)
```

d)

```
1 Aucune id e...
```

Exercice 52 Enoncé à ajouter, graphes et tests à faire

Corrigé 52 a)

```
1 def est_atteinte_bernoulli(k,n,p):
2     return k <= np.sum(np.array([np.binomial(1,p)+1 for _ in range(n)]))
```

b)

```
1 def proba_e_bernoulli(k,n,p):
2     cmpt = 0
3     for _ in range(10000):
4         if est_atteinte_bernoulli(k,n,p):
5             cmpt += 1
6     return cmpt / 10000
7
8 print("La probabilit de Ek vaut",proba_e_bernoulli(k,n,p)," et l'inverse de l'esperance vaut",1/(p+1))
```

c)

```
1 def est_atteinte_geometrique(k,n,p):
2     return k <= np.sum(np.array([np.geometric(p) for _ in range(n)]))
3
4 def proba_e_geometrique(k,n,p):
5     cmpt = 0
6     for _ in range(10000):
7         if est_atteinte_geometrique(k,n,p):
8             cmpt += 1
9     return cmpt / 10000
10
11 print("La probabilit de Ek vaut",proba_e_geometrique(k,n,p)," et l'inverse de l'esperance vaut",p)
```

11 Equations différentielles

Exercice 53 On définit sur $M_n(\mathbb{C})$ la norme : $\forall M = [m_{i,j}]_{1 \leq i,j \leq n} \in M_n(\mathbb{C}), \|M\| = \max_{1 \leq i \leq n} \left(\sum_{j=1}^n |m_{i,j}| \right)$.

1. En PYTHON, écrire une fonction représentant cette norme.

2. Soit $\phi : X \mapsto AX + XB$ définie sur $M_n(\mathbb{C})$, avec $A, B \in M_n(\mathbb{C})$. Écrire une fonction en PYTHON, qui prend en argument A et B et renvoie la matrice de ϕ dans la base canonique de $M_n(\mathbb{C})$.

Corrigé 53 1.

```
1 def norme(m):
2     return np.amax(np.sum(np.abs(m)[i] for i in range(m.shape[0])))
```

2.

```
1 def phi(x,a,b):
2     return np.dot(a,x) + np.dot(x,b)
3
4 def matrice_phi(a,b): # La matrice dans la base canonique est la transpose de la matrice dont toutes les
5     lignes sont phi(E11), phi(E12), ...
6     matrice = []
7     n = a.shape[0]
8     for i in range(n**2):
9         x = np.array([0]*n**2) # On cree une matrice elementaire sous la forme d'un tableau que l'on
10            redimensionne en matrice n*n
11         x[i]=1
12         matrice.append(phi(x.reshape((n,n)),a,b).reshape(n**2)) # On redimensionne la matrice obtenue en
13            tableau de taille n**2
14     return np.transpose(np.array(matrice))
```

Exercice 54 On pose pour tout $n \in \mathbb{N}$ et pour tout $x \in \mathbb{R}$: $J_n(x) = \int_0^{2\pi} \cos(x \sin(t) - nt) dt$.

On appelle zéros de J_n tout $x_0 \in \mathbb{R}$ tel que $J_n(x_0) = 0$.

1) Python : Écrire une fonction pour définir J_n et afficher J_n pour $x \in [0, 20]$ et $n \in \llbracket 0, 3 \rrbracket$.

On admet que J_n s'annule sur tout segment de longueur 1.

2) Python : définir une fonction `zeros(n,a)` qui renvoie les listes des ordonnées de zéros de J_n sur $[0, a]$.

Corrigé 54 1)

```

1 def j(x,n):
2     def f(t):
3         return np.cos(x * np.sin(t) - n*t)
4     return integr.quad(f,0,2*np.pi)[0]
5
6 j = np.vectorize(j)
7 X = np.arange(0,20,0.01)
8 Y0 = j(X,0)
9 Y1 = j(X,1)
10 Y2 = j(X,2)
11 Y3 = j(X,3)
12 plt.plot(X,Y0,label="J0",color="blue")
13 plt.plot(X,Y1,label="J1",color="red")
14 plt.plot(X,Y2,label="J2",color="green")
15 plt.plot(X,Y3,label="J3",color="orange")
16 plt.legend()
17 plt.grid()
18 plt.show()

```

et on obtient le graphique suivant :

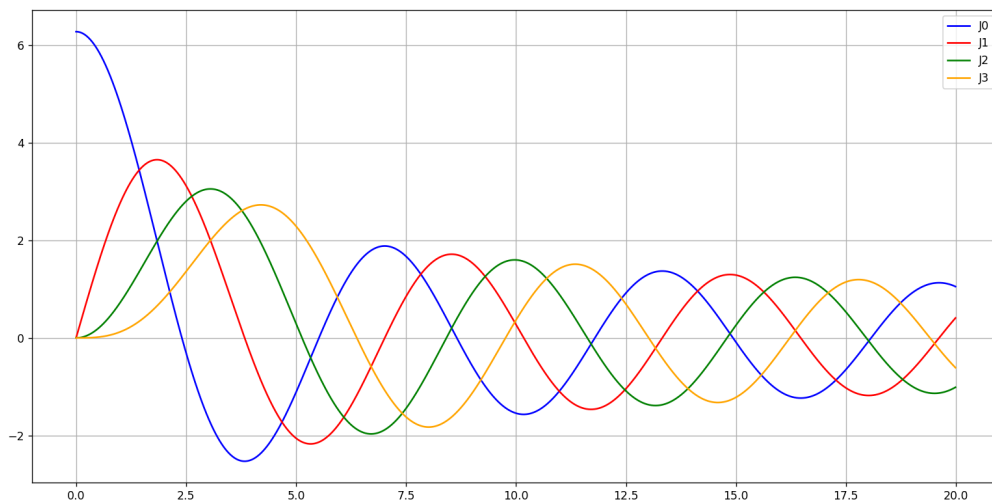


FIGURE 20 – Tracé de $J_n(x)$ pour différentes valeurs de n

2)

```

1 def zeros(n,a):
2     liste_zeros = []
3     for x in np.arange(0.01,a,0.01):
4         if j(x,n) * j(x-0.01,n) <= 0: # Si la fonction a change de signe, alors elle s'est annulée d'après
5             le TVI
6             liste_zeros.append(x)
7     return np.array(liste_zeros)

```

Exercice 55 On note, pour tout $n \in \mathbb{N}^*$, $a_n = \frac{n^{n-1}}{n!}$. On note $W(x)$ la somme de la série $\sum a_n x^n$.

2) Tracer sur Python les 300 premières sommes partielles.

4) On note, pour tout $p \in \mathbb{N}^*$, $S_p = \sum_{k=0}^p \binom{p}{k} (-1)^k k^{p-1}$. Calculer avec Python les 10 premiers termes de la suite $(S_p)_{p \in \mathbb{N}^*}$.

Corrigé 55 2)

```

1 def a(n):
2     return n**(n-1) / np.math.factorial(n)
3
4 X = np.arange(-0.4, 0.4, 0.01)
5 somme_partielles = np.zeros((300, len(X)))
6 for n in range(1, 301):
7     somme_partielles[n - 1] = somme_partielles[n - 2] + a(n) * (X ** n) if n > 1 else a(n) * X
8
9 for n in range(10, 301, 30):
10    plt.plot(X, somme_partielles[n - 1], label=f'N = {n}')
11 plt.legend()
12 plt.grid()
13 plt.show()

```

et on obtient le graphique suivant :

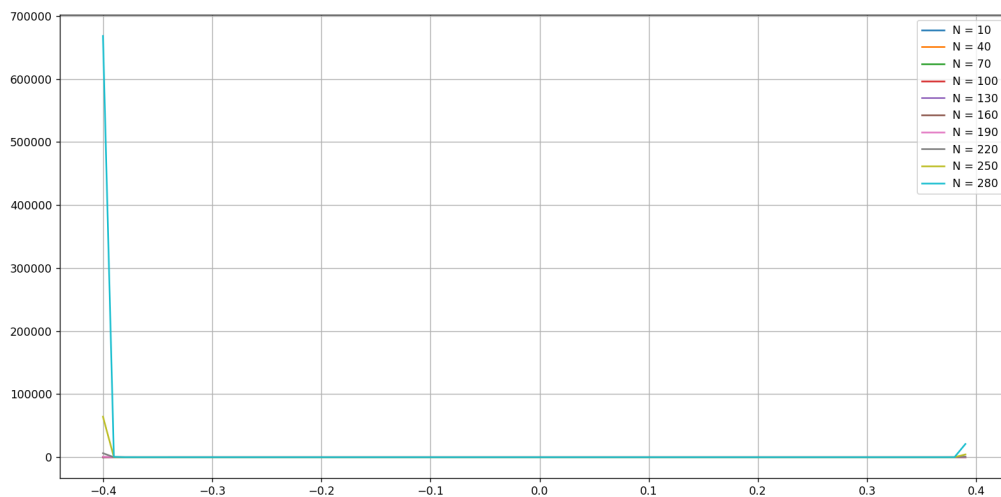


FIGURE 21 – Tracé des 300 premières sommes partielles

4)

```

1 def binom(k,n):
2     return np.math.factorial(n) // (np.math.factorial(k) * np.math.factorial(n-k))
3
4 def s(p):
5     return np.sum(np.array([binom(k, p) * (-1)**k * (k**(p-1) if k != 0 else 1) for k in range(p + 1)]))
6
7 for i in range(1,11):
8     print("S",i,"=",s(i))

```

12 Intégrales à paramètre

Exercice 56 Tracer la courbe représentative de la fonction f définie par $f(x) = \int_0^{2\pi} \cos(x \sin(t)) dt$.

```
1 def f(x):
2     def g(t):
3         return np.cos(x * np.sin(t))
4     return integr.quad(g,0,np.pi/2)[0]
5
6 f = np.vectorize(f)
7 X = np.arange(-150,150,0.1)
8 Y = f(X)
9 plt.plot(X,Y)
10 plt.title("Trace de f")
11 plt.show()
```

et on obtient le graphique suivant :

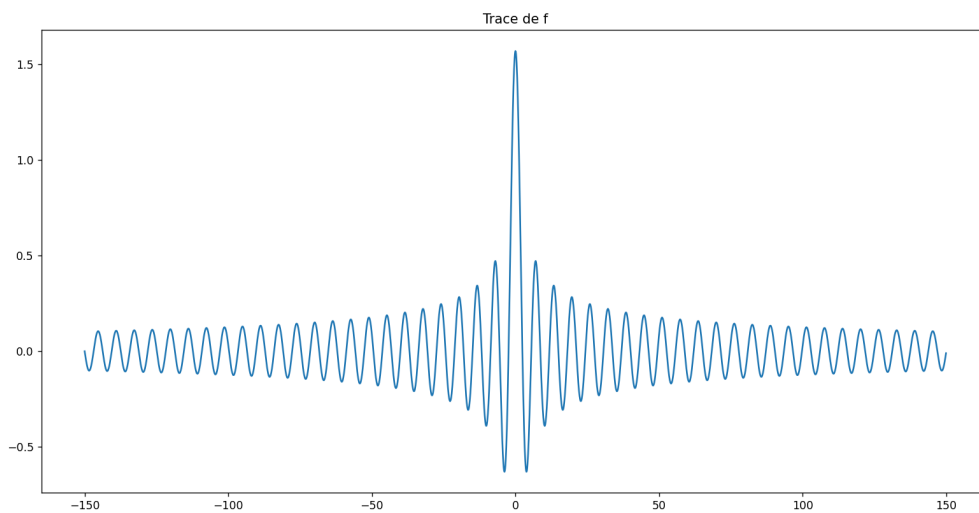


FIGURE 22 – Tracé de la fonction f

Exercice 57 1. a) Écrire un programme Python qui pour un entier naturel non nul n associe le nombre de diviseurs positifs $d(n)$.

b) On pose : $\forall x \in]-1, 1[$, $\forall n \in \mathbb{N}^*$, $u_n(x) = \frac{x^n}{1-x^n}$ et $f(x) = \sum_{n=1}^{+\infty} u_n(x)$. Écrire un programme Python qui donne une approximation de $f(x)$ et comparer à $\sum_{n=1}^{+\infty} d(n)x^n$. Conjecture?

Corrigé 57 1. a)

```
1 def d(n):
2     cmpt = 0
3     for d in range(1,n+1):
4         if n%d==0:
5             cmpt += 1
6     return cmpt
```

b)

```
1 def u(n,x):
2     return x**n/(1-x**n)
3
4 def f(x,N):
5     return np.sum([u(n,x) for n in range(1,N)])
6
7 def g(x,N):
8     return np.sum([d(n)*x**n for n in range(1,N)])
9
10 f = np.vectorize(f)
11 g = np.vectorize(g)
12 N = 500
13 X = np.arange(-0.99,0.99,0.01)
14 Y = f(X,N)
15 Z = g(X,N)
16 plt.plot(X,Y,label="f(x)",color="blue")
17 plt.plot(X,Z,label="g(x)",color="orange")
18 plt.legend()
19 plt.show()
```

et on obtient le graphique suivant :

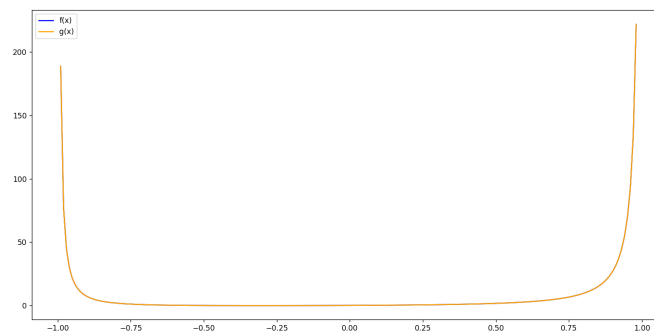


FIGURE 23 – Tracé de la fonction f et de la fonction g

Les deux graphes se superposent : on peut conjecturer que $f = g$ pour tout $x \in]-1 ; 1[$.

Exercice 58 Soit $\beta > 0$. On pose, pour $x \in \mathbb{R}$: $C(x) = \int_0^{+\infty} \frac{\cos(xt)}{1+t^2} dt$ et $I_\beta(x) = \int_0^{+\infty} \frac{\cos(xt)e^{-\beta t}}{1+t^2} dt$.

a) Définir une fonction qui calcule C.

b) Représenter $x \mapsto C(x)e^x$, conjecturer une expression de C sur $\mathbb{R}$.

Corrigé 58 a)

```
1 def c(x):
2     def f(t):
3         return np.cos(x*t) / (1 + t**2)
4     return integr.quad(f,0,np.inf)[0]
```

b)

```
1 def g(x):
2     return c(x) * np.exp(x)
3
4 g = np.vectorize(g)
5
6 X = np.arange(-10,10,0.01)
7 Y = g(X)
8 plt.plot(X,Y)
9 plt.show()
```

et on obtient le graphique suivant :

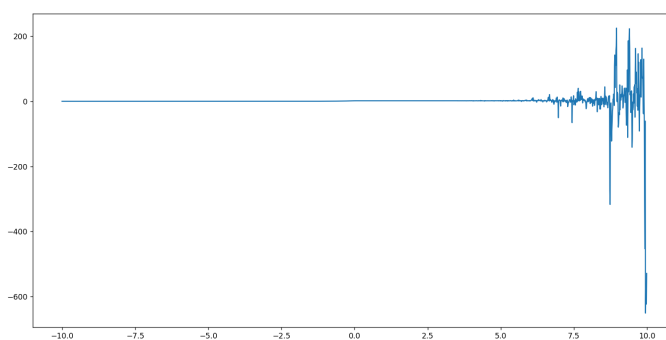


FIGURE 24 – Tracé de la fonction $C(x)e^x$

On peut conjecturer que C est la fonction nulle sur $\mathbb{R}$.

13 Structures algébriques, polynômes et arithmétique

Exercice 59 On appelle "faux premier" tout entier $n \geq 2$ non premier vérifiant la propriété suivante :

$$(C) : \forall a \in \mathbb{Z}, a \wedge n = 1 \Rightarrow a^{n-1} \equiv 1[n]$$

Notons que la propriété (C) est équivalente à la propriété suivante :

$$(C') : \forall a \in \llbracket 2, n \rrbracket, a \wedge n = 1 \Rightarrow a^{n-1} \equiv 1[n].$$

L'objectif de l'exercice est de démontrer que les faux premiers sont les entiers $n \geq 2$ vérifiant les trois conditions suivantes :

- n n'est pas premier.
 - n n'a aucun facteur premier multiple.
 - Pour tout facteur premier p de n , on a que $p - 1$ divise $n - 1$.
1. Montrer que tout faux premier est impair. Indication : on pourra considérer l'entier -1 .
 2. Coder en python une fonction `pgcd(a, b)` qui renvoie le pgcd des entiers a et b .
 3. Coder en python une fonction `crible(n)` qui une liste L de Booléens, de taille n , et telle que l'élément d'indice k de L soit `True` si k n'est pas premier, et `False` sinon. Afficher tous les faux premiers inférieurs à 10000

Corrigé 59 2.

```
1 def pgcd(a,b):
2     if b==0:
3         return a
4     return pgcd(b,a%b)
```

3.

```
1 def crible(n):
2     res = np.full(n+1, False) # Permet d'obtenir un array rempli de False
3     # Initialisation pour 0 et 1
4     res[0] = True
5     if n==0:
6         return res
7     res[1] = True
8     for i in range(2,n+1):
9         for j in range(i*2,n+1,i): # On commence a 2i et on augmente de i a chaque iteration (donc 3i, 4i
10             ...
11             res[j] = True
12     return res
```

```
1 def facteurs_preiers(a): # Renvoie un array vide si a a un facteur premier multiple
2     premiers = crible(a)
3     facteurs = []
4     for i in range(a):
5         if not premiers[i] and a%i == 0: # Si i est premier et divise a
6             if a%(i*i) == 0: # Si i est un facteur premier multiple
7                 return np.array([])
8             facteurs.append(i)
9     return np.array(facteurs)
10
11 faux_preiers = []
12 for i in range(3,10000,2): # Les faux premiers sont impairs d'apres la question 1
13     facteurs = facteurs_preiers(i)
14     taille = np.shape(facteurs)[0]
15     est_faux_preier = True
16     if taille==0: # Si la liste est vide, c'est qu'il est premier ou qu'il a un facteur premier multiple
17         donc on arrete d'etudier ce nombre
18         est_faux_preier = False
19     for j in range(taille):
20         if (i-1)%(facteurs[j]-1) != 0: # Condition 3
21             est_faux_preier = False
22     if est_faux_preier:
23         faux_preiers.append(i)
24 faux_preiers = np.array(faux_preiers)
```

Exercice 60 Une permutation σ sera représentée par une liste Python : $[\sigma(1), \dots, \sigma(n)]$

On appelle Dérangement toute permutation $\sigma \in \mathfrak{S}_n$ telle que son support soit $\llbracket 1, n \rrbracket$ (c'est-à-dire sans point fixe). On notera par la suite d_k le nombre de dérangements de $\mathfrak{S}_k$

Partie Python :

1. Programmer une fonction *est_derangement* prenant en entrée une permutation σ déterminant si σ est un dérangement.
2. Programmer une fonction *permutations* prenant en entrée un entier n renvoyant toute les permutations de $\mathfrak{S}_n$ (On pourra procéder de manière récursive).
3. Programmer une fonction *derangements* prenant en entrée un entier n renvoyant le nombre de dérangements de $\mathfrak{S}_n$.

Corrigé 60 1.

```
1 def est_derangement(sigma):
2     n = sigma.shape[0]
3     # On verifie s'il contient une fois chaque valeur de 1 à n
4     verif = np.full(n, False)
5     for i in range(n):
6         # On verifie en plus s'il ne possède pas de point fixe
7         if sigma[i]-1 == i:
8             return False
9         verif[sigma[i]-1] = True
10    if not np.array_equal(verif, np.full(n, True)):
11        return False
12    return True
```

2.

```
1 toutes_permutations = []
2 def aux(n, perm, used):
3     if len(perm) == n:
4         toutes_permutations.append(np.array(perm)) # On transforme en array, ce qui retire les effets de
5         bord
6     else:
7         for i in range(1, n + 1):
8             if not used[i]:
9                 used[i] = True
10                perm.append(i)
11                aux(n, perm, used)
12                perm.pop()
13                used[i] = False
14 def permutations(n):
15     global toutes_permutations
16     toutes_permutations = []
17     used = np.full(n+1, False)
18     aux(n, [], used)
19     return toutes_permutations
```

3.

```
1 def derangements(n):
2     cmpt = 0
3     global toutes_permutations
4     permutations(n)
5     for sigma in toutes_permutations:
6         if est_derangement(sigma):
7             cmpt += 1
8     return cmpt
```

Exercice 61 Soit $n \in \mathbb{N}$. On note D_n l'ensemble des diviseurs positifs de n . Soit $\sigma(n) = \sum_{d \in D_n} d$. On note Z l'ensemble des $n \in \mathbb{N}$ tels que $D_n = A \cup B$ avec A et B disjoints tels que $\sum_{d \in A} d = \sum_{d \in B} d$.

1. Écrire une fonction Python qui renvoie la liste des diviseurs positif d'un entier n .
2. Écrire une fonction Python qui renvoie la liste des entiers n entre 1 et 100 appartenant à Z .

Commentaires divers : 2)a) Pour le code, une fonction renvoyant la liste des sous-listes d'une liste était donnée.

Corrigé 61 1.

```
1 def diviseurs(n):
2     res = []
3     for i in range(1,n+1):
4         if n%i == 0:
5             res.append(i)
6     return np.array(res)
```

2.

```
1 # D'après le commentaire, une fonction semblable etait donnee
2 def aux(l, sl, liste, index):
3     if index == len(l):
4         liste.append(np.array(sl))
5     else:
6         aux(l, sl, liste, index + 1)
7         sl.append(l[index])
8         aux(l, sl, liste, index + 1)
9         sl.pop()
10
11 def liste_sous_listes(l):
12     liste = []
13     aux(l, [], liste, 0)
14     return liste
15
16 def appartient_Z():
17     res = []
18     for n in range(1,101):
19         div = diviseurs(n)
20         sous_listes = liste_sous_listes(div)
21         # On verifie pour chaque sous-liste si elle correspond un partitionnement qui convient
22         for sl in sous_listes:
23             cmpt_a = 0
24             cmpt_b = 0
25             index = 0
26             for i in range(len(div)):
27                 if index < len(sl) and div[i] == sl[index]:
28                     cmpt_a += div[i]
29                     index += 1
30             else:
31                 cmpt_b += div[i]
32             if cmpt_a == cmpt_b:
33                 res.append(n)
34                 break # Cela ne sert a rien de continuer a explorer les sous listes si on en a trouve une
35                     # qui convient
36     return np.array(res)
```

Exercice 62 Pour tout entier $n \in \mathbb{N}^*$, on note $\mathbb{U}_n$ l'ensemble des racines n -ièmes de l'unité et Π_n l'ensemble des racines primitives de l'unité (c'est-à-dire des générateurs de $\mathbb{U}_n$).

Après avoir fixé $n \in \mathbb{N}^*$, on notera par la suite $\omega = e^{\frac{2i\pi}{n}}$. Pour p un nombre premier, on notera $\Phi_p = \prod_{\xi \in \Pi_n} (X - \xi)$

b) Sur Python, afficher les Φ_p pour p premier de 2 à 13.

Indication(s) fournie(s) par l'examineur pendant l'épreuve : L'examineur a clarifié pour moi le sens de la question 1. b), il était attendu qu'on affiche non pas la courbe représentative des polynômes, mais leurs formules avec les coefficients.

Corrigé 61 b)

```
1 def phi(p):
2     res = Polynomial([-1,1]) # X - xi pour xi = 1
3     for k in range(1,p):
4         res = res * Polynomial([-np.exp(2j*k*np.pi/p),1])
5     return res
6
7 print("p = 2:",phi(2))
8 print("p = 3:",phi(3))
9 print("p = 5:",phi(5))
10 print("p = 7:",phi(7))
11 print("p = 11:",phi(11))
12 print("p = 13:",phi(13))
```

Exercice 63 Soit n un entier naturel et $V = \mathbb{C}_n[X]$.

Pour $g = \begin{pmatrix} a & b \\ c & d \end{pmatrix}$, on note pour tout polynôme P , $P_g = (cX + a)^n P\left(\frac{-dX+b}{-cX+a}\right)$.

Soit ϕ_g l'application qui à P associe P_g .

b) Écrire une fonction `action(g, P)` qui retourne P_g (on pourra utiliser le module `Polynomial`). On donnait un exemple et le résultat attendu pour vérifier le fonctionnement.

c) Écrire une fonction `mat(g)` qui renvoie la matrice de ϕ_g dans la base canonique. La lancer sur S , U et T , calculer les valeurs propres à chaque fois (matrices simples données au début, avec que des 0, 1 et -1)

Corrigé 63 b)

```
1 n=4
2 def action(g,P):
3     polynome_puissance = Polynomial([g[0,0],-g[1,0]])**n # Pour elever un polynome a la puissance n, il faut
        utiliser **n
4     polynome_compose = Polynomial([g[0,1],-g[1,1]]) // Polynomial([g[0,0],-g[1,0]]) # Attention: pour
        diviser deux polynomes, il faut utiliser //
5     return polynome_puissance * P(polynome_compose)
```

c)

```
1 def mat(g):
2     # La transposee de la matrice voulue est la matrice dont toutes les lignes sont action(g,1), action(g,X)
        , ..., action(g,X^n)
3     matrice = []
4     for i in range(n+1):
5         matrice.append(action(g,Polynomial([0]*i+[1])).coef)
6     return np.transpose(np.array(matrice))
7
8 alg.eigvals(mat(g))
```

Exercice 64 Soit $p > 2$ un nombre premier : on sait alors que $(\mathbb{Z}/p\mathbb{Z})$ est un corps. Pour $k \in \mathbb{Z}$, on note $\bar{k}$ son représentant modulo p . On pose alors $(\mathbb{Z}/p\mathbb{Z})^* = (\mathbb{Z}/p\mathbb{Z}) \setminus \{\bar{0}\}$.

On admet que pour tout polynôme P de degré d dans un corps, P admet au plus d racines dans ce corps.

On note enfin $C = \{x^2 \mid x \in (\mathbb{Z}/p\mathbb{Z})^*\}$.

1. a) Écrire une fonction Python notée *ensemble_c* qui prend comme paramètre un nombre premier p , et qui renvoie une liste constituée de tous les $k \in \llbracket 1, p \rrbracket$ vérifiant $k = x^2$, où $x \in (\mathbb{Z}/p\mathbb{Z})^*$.

b) Pour tous les nombres premiers entre 3 et 100, calculer $\text{card}(C)$ avec Python. Que peut-on conjecturer?

Corrigé 64 1. a)

```
1 def ensemble_c(p):
2     res = []
3     deja_present = np.full(p, False) # Permet d'obtenir un array rempli de False
4     for x in range(1, p):
5         if not deja_present[x*x%p]:
6             res.append(x*x%p)
7             deja_present[x*x%p] = True
8     return np.array(res)
```

b)

```
1 def crible(n):
2     res = np.full(n+1, True) # Permet d'obtenir un array rempli de True
3     # Initialisation pour 0 et 1
4     res[0] = False
5     if n==0:
6         return res
7     res[1] = False
8     for i in range(2, n+1):
9         for j in range(i*2, n+1, i): # On commence a 2i et on augmente de i a chaque iteration (donc 3i, 4i ...)
10            res[j] = False
11     return res
12
13 est_premier = crible(100)
14 for i in range(100):
15     if est_premier[i]:
16         print("p =", i, "et card(c) =", ensemble_c(i).shape[0])
```

On peut conjecturer que le cardinal de C vaut la moitié de p (arrondi à l'inférieur)

Exercice 65 Soit G un sous groupe de $O_n(\mathbb{R})$, on définit la trace de G par $\text{tr}(G) = \{\text{tr}(A), A \in G\}$. On dit que G est d'exposant fini s'il existe $m \in \mathbb{N}^*$ tel que pour tout $A \in G$, $A^m = I_n$. On note $R(\theta)$ la matrice de rotation d'angle θ .

1. a) Écrire une fonction sur Python qui renvoie la matrice de rotation d'angle θ .

b) Soit deux matrices A et B , écrire une fonction sur Python qui calcule le groupe engendré par les matrices A et B .

Pour que la complexité ne soit pas très élevée : si le groupe engendré possède plus de 100 éléments ou est infini, l'algorithme ne devra renvoyer que les 100 premières matrices calculées. pour parcourir la liste L de matrices construites, on utilisera `for C in L[:]` plutôt que `for C in L`. (également une autre consigne dont je ne me rappelle plus)

c) Vérifier que le groupe engendré par $R_{\frac{\pi}{3}}$ et $R_{\frac{\pi}{5}}$ est bien fini et possède 30 éléments.

Corrigé 65 1. a)

```
1 def rotation(theta):
2     return np.array([[np.cos(theta), -np.sin(theta)], [np.sin(theta), np.cos(theta)]])
```

b)

```
1 def sous_groupe_engendre(a,b):
2     matrices_simples = [a,b]
3     sous_groupe = [np.eye(a.shape[0])]
4     if alg.det(a) != 0:
5         matrices_simples.append(alg.inv(a)) # On ajoute l'inverse de a si elle est inversible
6     if alg.det(b) != 0:
7         matrices_simples.append(alg.inv(b)) # On ajoute l'inverse de b si elle est inversible
8     cmpt = 0
9     nb_matrices_ajoutees = 1 # Representant le nombre de matrices ajoutes lors de la derniere iteration
10    while nb_matrices_ajoutees > 0 and len(sous_groupe)<100:
11        for i in range(nb_matrices_ajoutees): # On ajoute les matrices simples a toutes les matrices
12            # generees a la derniere iteration
13            for m in matrices_simples:
14                matrice = np.dot(sous_groupe[-1-i],m)
15                matrice_est_nouvelle = True
16                for n in sous_groupe[:]:
17                    if np.allclose(n,matrice, rtol=1e-10, atol=1e-10): #np.array_equal() ne fonctionne pas
18                        # ici a cause de la manipulation de flottants
19                        matrice_est_nouvelle = False
20                        break
21                if matrice_est_nouvelle:
22                    sous_groupe.append(matrice)
23                    cmpt += 1
24            nb_matrices_ajoutees = cmpt
25            cmpt = 0
26    return np.array(sous_groupe[:100])
```

c)

```
1 print(sous_groupe_engendre(rotation(np.pi/3),rotation(np.pi/5)).shape[0])
```

Exercice 66 Soit $n \in \mathbb{N}^*$. On pose $\sigma(n) = \sum_{d|n} d$, et $\tau(n) = \text{Card}\{d \in \mathbb{N} / d|n\}$.

1. Coder σ et τ en Python.
2. Numériquement, donner les $k \in \llbracket 1, 1000 \rrbracket$ tels que $\sigma \circ \tau(n) = n$, puis ceux tels que $\tau \circ \sigma(n) = n$.

Corrigé 66 1.

```
1 def sigma(n):
2     cmpt = 0
3     for d in range(1,n+1):
4         if n%d==0:
5             cmpt += d
6     return cmpt
7
8 def tau(n):
9     cmpt = 0
10    for d in range(1,n+1):
11        if n%d==0:
12            cmpt += 1
13    return cmpt
```

2.

```
1 sigma_tau = []
2 tau_sigma = []
3 for k in range(1,1001):
4     if sigma(tau(k))==k:
5         sigma_tau.append(k)
6     if tau(sigma(k))==k:
7         tau_sigma.append(k)
8 sigma_tau = np.array(sigma_tau)
9 tau_sigma = np.array(tau_sigma)
```

Exercice 67 1. a) Coder $\binom{n}{k}$.

b) On note $P(n)$ l'ensemble des nombres premiers inférieurs ou égaux à n et P_n le produit des éléments de $P(n)$.

Coder sur Python une fonction qui renvoie P_n , puis calculer $P_{\binom{4}{2}}$ et $P_{\binom{6}{3}}$.

Corrigé 67 1. a)

```
1 def binom(k,n):
2     return np.math.factorial(n) // (np.math.factorial(k) * np.math.factorial(n-k))
```

b)

```
1 def p(n):
2     cmpt = 1
3     res = np.full(n+1, True) # Permet d'obtenir un array rempli de True
4     for i in range(2,n+1):
5         for j in range(i*2,n+1,i): # On commence a 2i et on augmente de i a chaque iteration (donc 3i, 4i
6             ...
7             res[j] = False
8         # Si i est premier, on multiplie
9         if res[i]:
10             cmpt *= i
11     return cmpt
12 print(p(binom(2,4)))
13 print(p(binom(3,6)))
```

Exercice 68 On fixe A un entier supérieur ou égal à 3. On note $a \text{ div } b$ et $a \text{ mod } b$ respectivement le quotient et le reste de la division euclidienne de a par b .

On définit la suite $(F_n)_{n \in \mathbb{N}}$ la suite telle que $F_0 = 0$, $F_1 = 1$ et $\forall n \in \mathbb{N}$, $F_{n+2} = F_{n+1} + F_n$.

On note : $\forall n \in \mathbb{N}$, $u_n = [A^{n^2+n} \bmod (A^{2n} - A^n - 1)] \bmod A^n$.

Utiliser Python pour calculer les vingt premiers termes de $(F_n)_{n \in \mathbb{N}}$ et de $(u_n)_{n \in \mathbb{N}}$. Que peut-on conjecturer?

```

1 def f(n):
2     res = np.zeros(n, dtype=int)
3     if n==0:
4         return res
5     res[1]=1
6     for i in range(2,n):
7         res[i] = res[i-1] + res[i-2]
8     return res
9
10 A = 3 # Exemple
11 def u(n):
12     res = np.zeros(n, dtype=int)
13     for i in range(n):
14         res[i] = (A**(i**2+i) % (A**(2*i) - A**i - 1)) % A**i
15     return res

```

On peut conjecturer que, peu importe la valeur de A , $f(n) = u(n)$

Exercice 69 1) On définit la famille $(F_n)_{n \in \mathbb{Z}}$ avec : $F_0 = 0, F_1 = 1$ et la relation : $\forall n \in \mathbb{Z}, F_{n+1} = F_n + F_{n-1}$. Coder une fonction renvoyant F_n pour n dans $\mathbb{Z}$. Afficher F_n pour $-10 \leq n \leq 10$.

3) Coder une fonction donnant le pgcd de deux entiers via l'algorithme d'Euclide, et le nombre de divisions euclidiennes effectuées. La tester avec des couples (F_{n+2}, F_{n+1}) pour $n > 0$. Commenter.

Corrigé 69 1)

```
1 def f(n):
2     val0 = 0
3     val1 = 1
4     for i in range(2,n+1):
5         temp = val1
6         val1 = val0 + temp
7         val0 = temp
8     for i in range(-n+1): # Il faut faire une iteration de plus pour que le r sultat soit dans val1
9         temp = val0
10        val0 = val1 - temp
11        val1 = temp
12    return val1
13
14 for i in range(-10,11):
15     print(f(i), end=" ; ")
```

3)

```
1 def aux(a,b,nb_div):
2     if b==0:
3         return a,nb_div
4     return aux(b,a%b,nb_div+1)
5
6 def pgcd(a,b):
7     res,cmpt = aux(a,b,0)
8     print("Nombre de divisions effectuees:",cmpt)
9     return a
10
11 for i in range(-10,11):
12     print(i,i+1, end=": ")
13     pgcd(f(i),f(i+1))
```

Le nombre de divisions effectuées augmente de 1 quand i augmente de 1.

On dirait que $\text{pgcd}(f(i),f(i+1))$ effectue i divisions (si $i > 0$)

14 Calcul différentiel

Exercice 70 On note, pour n dans $\mathbb{N}^*$, $\mathfrak{S}_n$ le groupe des permutations de $\{1, 2, \dots, n\}$. Pour M notée $(m_{i,j})_{1 \leq i,j \leq n}$ matrice carrée d'ordre n à coefficients réels, le permanent de M , noté $\text{Per}(M)$, est le nombre $\text{Per}(M) = \sum_{\sigma \in \mathfrak{S}_n} \prod_{i=1}^n m_{i,\sigma(i)}$.

On dit que M notée $(m_{i,j})_{1 \leq i,j \leq n}$ dans $M_n(\mathbb{R})$ est bistochastique si elle vérifie : $\forall i, j \in \{1, 2, \dots, n\}^2, m_{i,j} \geq 0, \forall i \in \{1, 2, \dots, n\}, \sum_{j=1}^n m_{i,j} = 1$ et $\forall j \in \{1, 2, \dots, n\}, \sum_{i=1}^n m_{j,i} = 1$. On pose $A = (\frac{1}{3})_{1 \leq i,j \leq 3}$.

1. a) Ecrire une fonction $\text{per}(M)$ qui calcule le permanent d'une matrice M carrée d'ordre 3. La vérifier sur la matrice A .

b) Soit $(a, b, c, d) \in \mathbb{R}^4$. Compléter si possible les coefficients de la matrice $\begin{pmatrix} a & b & . \\ c & d & . \\ . & . & . \end{pmatrix}$ pour qu'elle soit bistochastique, et

donner la condition sur (a, b, c, d) pour que ce soit possible.

Programmer une fonction Python qui renvoie une matrice bistochastique aléatoire d'ordre 3 (on rappelle que la fonction `numpy.random.random()` renvoie un flottant pseudo-aléatoire suivant une loi uniforme sur $[0, 1]$).

c) Écrire une fonction qui tire N matrices bistochastiques aléatoires d'ordre 3 et renvoie celle de permanent minimal (ainsi que son permanent). L'utiliser pour $N = 10000$. Que peut-on conjecturer?

Corrigé 70 1. a)

```
1 s_3 = np.array([np.array([1,2,3]),np.array([1,3,2]),np.array([2,1,3]),np.array([2,3,1]),np.array([3,1,2]),np
2               .array([3,2,1])]) # Liste des permutations de S_3
3
4 def perm(m):
5     cmpt = 0
6     for sigma in s_3:
7         temp = 1
8         for i in range(3):
9             temp *= m[i,sigma[i]-1]
10        cmpt += temp
11    return cmpt
12
13 a = np.full((3,3),1/3)
14 print(perm(a))
```

b) En omettant les détails, si on a $0.5 \leq a+b \leq 1, 0.5 \leq c+d \leq 1$ et de même pour les colonnes, on a un unique résultat pour les valeurs de chaque point (et le coin en bas à droite doit avoir la même valeur pour la colonne et pour la ligne).

```
1 def bistochastique():
2     matrice = np.zeros((3,3))
3     alea = (1 + rd.random()) / 2 # On genere un nombre aleatoire entre 0.5 et 1
4     matrice[0,0] = alea - (alea * rd.random()) # On genere un nombre aleatoire inferieur au nombre precedent
5     matrice[0,1] = alea - matrice[0,0] # On complete b pour que a+b = alea
6     matrice[0,2] = 1 - alea # Il ne nous reste qu'une unique facon de completer la ligne
7     alea = (1 + rd.random()) / 2 # On recommence mais pour la premiere colonne
8     matrice[1,0] = alea - matrice[0,0] # La valeur de a est deja fixee donc il ne reste plus que ca a faire
9     matrice[2,0] = 1 - alea
10    alea = (1 + rd.random()) / 2 # On complete la deuxieme ligne
11    matrice[1,1] = alea - matrice[1,0]
12    matrice[1,2] = 1 - alea
13    matrice[2,1] = 1 - matrice[1,1] - matrice[0,1]
14    matrice[2,2] = 1 - matrice[2,0] - matrice[2,1]
15    return matrice
```

c) On peut conjecturer que la matrice A est la matrice de permanent minimal

```
1 def permanent_minimum(N):
2     matrice_mini = bistochastique()
3     perm_mini = perm(matrice_mini)
4     for _ in range(N-1):
5         matrice = bistochastique()
6         if perm(matrice) < perm_mini:
7             perm_mini = perm(matrice)
8             matrice_mini = matrice
9     return matrice_mini, perm_mini
```